

Structured Training Program: Networking**Week 4***Module 7: Layer 3 Session 3 & Module 8: Layer 4 – TCP, UDP, NAT and Firewall*

1. Try Test-Connection/Test-NetConnection and nslookup commands for the below websites
www.google.com, www.facebook.com, www.amazon.com, www.github.com, www.cisco.com

Using Test-Connection on the following websites (with different available options)

a) To check based on port.

```

PS C:\Users\Mystery Chap> Test-NetConnection www.google.com -port 443

ComputerName      : www.google.com
RemoteAddress     : 2001:4860:4827:7700::
RemotePort        : 443
InterfaceAlias     : Wi-Fi
SourceAddress     : 2401:4900:ca7c:ae3f:11ac:dde0:f67a:b845
TcpTestSucceeded  : True
  
```

b) To check Based on common TCP application layer protocol

```

PS C:\Users\Mystery Chap> Test-NetConnection www.facebook.com -CommonTCP HTTP

ComputerName      : www.facebook.com
RemoteAddress     : 2a03:2880:f36a:1:face:b00c:0:25de
RemotePort        : 80
InterfaceAlias     : Wi-Fi
SourceAddress     : 2401:4900:ca7c:ae3f:11ac:dde0:f67a:b845
TcpTestSucceeded  : True

PS C:\Users\Mystery Chap> Test-NetConnection www.facebook.com -CommonTCP SMB
WARNING: TCP connect to (2a03:2880:f334:1:face:b00c:0:25de : 445) failed
WARNING: TCP connect to (57.144.54.1 : 445) failed

ComputerName      : www.facebook.com
RemoteAddress     : 2a03:2880:f334:1:face:b00c:0:25de
RemotePort        : 445
InterfaceAlias     : Wi-Fi
SourceAddress     : 2401:4900:ca7c:ae3f:11ac:dde0:f67a:b845
PingSucceeded     : True
PingReplyDetails (RTT) : 44 ms
TcpTestSucceeded  : False
  
```

c) To check the path it takes to reach from local machine

```

PS C:\Users\Mystery Chap> Test-NetConnection www.github.com -TraceRoute

ComputerName      : www.github.com
RemoteAddress     : 20.207.73.82
InterfaceAlias     : Wi-Fi
SourceAddress     : 10.161.138.50
PingSucceeded     : True
PingReplyDetails (RTT) : 71 ms
TraceRoute        : 10.161.138.158
                   0.0.0.0
                   192.168.116.148
                   192.168.116.177
                   0.0.0.0
                   61.246.213.105
                   116.119.161.149
                   198.200.130.17
                   51.10.44.156
                   51.10.23.169
                   0.0.0.0
                   0.0.0.0
                   104.44.22.22
                   0.0.0.0
                   0.0.0.0
                   0.0.0.0
                   0.0.0.0
                   0.0.0.0
                   20.207.73.82
  
```

d) To control the amount of information shown.

```

C:\WINDOWS\system32\cmd.exe  Windows PowerShell
PS C:\Users\Mystery Chap> Test-NetConnection www.cisco.com -InformationLevel Detailed

ComputerName           : www.cisco.com
RemoteAddress          : 2600:140f:5e00:482::b33
NameResolutionResults  : 2600:140f:5e00:482::b33
                        : 2600:140f:5e00:480::b33
                        : 23.41.120.121
InterfaceAlias         : Wi-Fi
SourceAddress          : 2401:4900:ca7c:ae3f:11ac:dde0:f67a:b845
NetRoute (NextHop)     : fe80::5c24:a3ff:fe8f:be4c
PingSucceeded          : True
PingReplyDetails (RTT) : 31 ms

PS C:\Users\Mystery Chap> Test-NetConnection www.cisco.com -InformationLevel quiet
True
PS C:\Users\Mystery Chap>

```

Using nslookup command to on different website

```

C:\WINDOWS\system32\cmd.exe  Windows PowerShell
PS C:\Users\Mystery Chap> nslookup www.google.com
Server: Unknown
Address: 10.161.138.158

Non-authoritative answer:
Name: www.google.com
Addresses: 2001:4860:4826:7700::
           2001:4860:482c:7700::
           2001:4860:482a:7700::
           2001:4860:4828:7700::
           2001:4860:482d:7700::
           2001:4860:4827:7700::
           2001:4860:482b:7700::
           2001:4860:4829:7700::
           142.251.153.119
           142.251.155.119
           142.251.156.119
           142.251.157.119
           142.251.154.119
           142.251.151.119
           142.251.150.119
           142.251.152.119

PS C:\Users\Mystery Chap> nslookup www.facebook.com
Server: Unknown
Address: 10.161.138.158

Non-authoritative answer:
Name: star-mini.c10r.facebook.com
Addresses: 2a03:2880:f36a:1:face:b00c:0:25de
           57.144.54.1
Aliases: www.facebook.com

PS C:\Users\Mystery Chap> nslookup www.amazon.com
Server: Unknown
Address: 10.161.138.158

Non-authoritative answer:
Name: e15316.dsca.akamaiedge.net
Addresses: 2600:140f:5e00:486::3bd4
           2600:140f:5e00:48a::3bd4
           23.41.122.94
Aliases: www.amazon.com
           tp.47cf2c8c9-frontier.amazon.com
           www.amazon.com.edgekey.net

PS C:\Users\Mystery Chap> nslookup www.github.com
Server: Unknown
Address: 10.161.138.158

Non-authoritative answer:
Name: github.com
Address: 20.207.73.82
Aliases: www.github.com

PS C:\Users\Mystery Chap> nslookup www.cisco.com
Server: Unknown
Address: 10.161.138.158

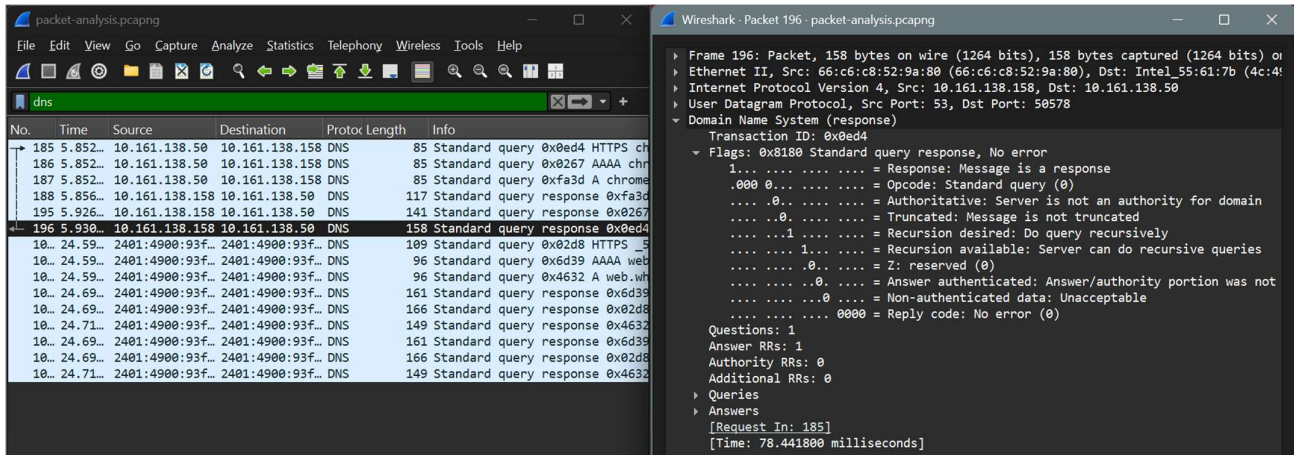
Non-authoritative answer:
Name: e2867.dsca.akamaiedge.net
Addresses: 2600:140f:5e00:482::b33
           2600:140f:5e00:480::b33
           23.41.120.121
Aliases: www.cisco.com
           www.cisco.com.akadns.net
           wwwds.cisco.com.edgekey.net
           wwwds.cisco.com.edgekey.net.globalredir.akadns.net

PS C:\Users\Mystery Chap>

```

2. Use Wireshark to capture and analyze DNS, TCP and UDP traffic and packet headers, packet flow, Options and flags

Analyzing DNS packet:



1. Packet Header, Option and Flags:

The DNS packet header is of size 12 bytes, consisting of the following,

- a) Transaction ID (16 bits): A unique 16-bit identifier generated by the client. The server copies this same ID into the response, allowing the client to match the response to a specific query.
- b) Flags (16 bits): Most critical part contains several components:
 - QR (Query/Response) (1 bit): 0 for query, 1 for response.
 - Opcode (4 bits): Indicates the type of message. 0 for standard query, 1 for inverse query (obsolete), 2 for server status, and 5 for dynamic update.
 - AA (Authoritative Answer) (1 bit): Set to 1 if the responding server is the authority for the domain.
 - TC (Truncated) (1 bit): Set to 1 if the message is longer than 512 bytes (UDP) and was truncated.
 - RD (Recursion Desired) (1 bit): Set by the client to request that the server recursively resolve the query.
 - RA (Recursion Available) (1 bit): Set by the server to indicate whether it supports recursion.
 - Z (Reserved) (3 bits): Reserved for future use; must be 0.
 - RCODE (Return Code) (4 bits): Indicates the status of the response:
 - 0: No Error (Success)
 - 1: Format Error
 - 2: Server Failure
 - 3: Name Error (NXDOMAIN - Domain does not exist)
 - 4: Not supported by server
 - 5: non execution of query by server due to policy reasons

- c) Number of questions (16 bits) – Specify the no. of questions in the question section
- d) Number of answer RRs (16 bits) – Specify the no. of answers in the answer section
- e) Number of authority RRs (16 bits) – Has a value of zero in request/query message and available only in response message. Gives info about no. of authoritative servers a domain name has.
- f) Number of additional RRs (16 bits) – Tells about the additional information that helps the resolver. Has a value of zero in request/query message and available only in response message

The things above this comprise the header.

- g) Question: This section contains the actual question being asked to the DNS server.
- h) Answer: This section contains the official response to the query, if available.
- i) Authority: This section provides information about which DNS servers are authoritative (official owners) for the domain
- j) Additional: This section provides extra information that is not directly requested but is related to the answer or authority sections.

2. DNS Packet Flow:

A DNS packet flow involves a client (browser) sending a UDP port 53 query to a recursive resolver, which then queries root, TLD, and authoritative servers to map a domain name to an IP address. The response travels back through these servers, usually being cached for future requests

- i. **User Request:** The user types a domain (e.g., example.com) into the browser.
- ii. **Recursive Query:** The browser sends a DNS query packet (UDP, port 53) to the recursive resolver (often provided by the ISP).
- iii. **Root Server Query:** The resolver queries a DNS root nameserver.
- iv. **TLD Server Query:** The root server directs the resolver to a Top-Level Domain (TLD) server (e.g., .com).
- v. **Authoritative Server Query:** The TLD server directs the resolver to the authoritative nameserver for example.com.
- vi. **Answer Retrieval:** The authoritative server returns the IP address to the resolver.
- vii. **Final Response:** The resolver sends the IP address back to the browser.

Observation on DNS:

- a. Used UDP and port 53 for communication.
- b. Since UDP is used the order of the packet is not preserved

No.	Time	Source	Destination	Protoc	Length	Info
185	5.852...	10.161.138.50	10.161.138.158	DNS	85	Standard query 0x0ed4 HTTP
186	5.852...	10.161.138.50	10.161.138.158	DNS	85	Standard query 0x0267 AAAA
187	5.852...	10.161.138.50	10.161.138.158	DNS	85	Standard query 0xfa3d A ch
188	5.856...	10.161.138.158	10.161.138.50	DNS	117	Standard query response 0x
195	5.926...	10.161.138.158	10.161.138.50	DNS	141	Standard query response 0x
196	5.930...	10.161.138.158	10.161.138.50	DNS	158	Standard query response 0x
10111	24.59...	2401:4900:93f...	2401:4900:93f...	DNS	109	Standard query 0x02d8 HTTP
10112	24.59...	2401:4900:93f...	2401:4900:93f...	DNS	96	Standard query 0x6d39 AAAA
10113	24.59...	2401:4900:93f...	2401:4900:93f...	DNS	96	Standard query 0x4632 A we
10120	24.69...	2401:4900:93f...	2401:4900:93f...	DNS	161	Standard query response 0x
10123	24.69...	2401:4900:93f...	2401:4900:93f...	DNS	166	Standard query response 0x
10124	24.71...	2401:4900:93f...	2401:4900:93f...	DNS	149	Standard query response 0x
10120	24.69...	2401:4900:93f...	2401:4900:93f...	DNS	161	Standard query response 0x
10123	24.69...	2401:4900:93f...	2401:4900:93f...	DNS	166	Standard query response 0x
10124	24.71...	2401:4900:93f...	2401:4900:93f...	DNS	149	Standard query response 0x

Analysing TCP packets:

The image displays the Wireshark network protocol analyzer interface. The left pane shows a list of captured packets, with packet 33 selected. The middle pane shows the details of the selected packet, which is a TCP segment. The right pane shows the raw packet data in hexadecimal and ASCII.

Packet List:

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000	2401:4900:93f...	2603:1040:5:3...	TLS...	102	Application Data
2	0.096	2603:1040:5:3...	2401:4900:93f...	TCP	74	443 → 60113 [ACK] Seq=1
3	0.125	2600:140f:3::...	2401:4900:93f...	TLS...	330	Application Data
4	0.126	2401:4900:93f...	2600:140f:3::...	TLS...	109	Application Data
5	0.131	2600:140f:3::...	2401:4900:93f...	TLS...	105	Application Data
6	0.180	2600:140f:3::...	2401:4900:93f...	TCP	74	443 → 59851 [ACK] Seq=1
7	0.185	2401:4900:93f...	2600:140f:3::...	TCP	74	59851 → 443 [ACK] Seq=1
13	1.327	2401:4900:93f...	2603:1040:5:3...	TLS...	105	Application Data
16	1.448	2603:1040:5:3...	2401:4900:93f...	TCP	74	443 → 60113 [ACK] Seq=1
17	2.283	2401:4900:93f...	2404:6800:400...	TCP	75	55077 → 443 [ACK] Seq=1
18	2.367	2404:6800:400...	2401:4900:93f...	TCP	90	443 → 55077 [ACK] Seq=1
21	2.598	10.161.138.50	13.89.179.14	TCP	55	65460 → 443 [ACK] Seq=1
22	2.959	13.89.179.14	10.161.138.50	TCP	66	443 → 65460 [ACK] Seq=1
23	3.100	10.161.138.50	52.175.136.182	TLS...	111	Application Data
24	3.257	10.161.138.50	52.123.168.136	TLS...	111	Application Data
25	3.264	52.175.136.182	10.161.138.50	TLS...	100	Application Data
26	3.305	10.161.138.50	52.175.136.182	TCP	54	49946 → 443 [ACK] Seq=1
27	3.470	52.123.168.136	10.161.138.50	TLS...	100	Application Data
28	3.470	2600:140f:720...	2401:4900:93f...	TCP	98	Application Data
29	3.470	2600:140f:720...	2401:4900:93f...	TCP	74	443 → 53290 [FIN, ACK] Seq=1
30	3.471	2401:4900:93f...	2600:140f:720...	TCP	74	53290 → 443 [ACK] Seq=1
31	3.471	2401:4900:93f...	2600:140f:720...	TCP	74	53290 → 443 [FIN, ACK] Seq=1
32	3.525	10.161.138.50	52.123.168.136	TCP	54	49947 → 443 [ACK] Seq=1
33	3.778	2401:4900:93f...	2600:140f:720...	TCP	74	[TCP Retransmission]
34	3.843	2600:140f:720...	2401:4900:93f...	TCP	74	443 → 53290 [ACK] Seq=1
35	5.153	2401:4900:93f...	2603:1063:200...	TCP	75	54015 → 443 [ACK] Seq=1
36	5.293	2401:4900:93f...	2603:1063:200...	TCP	75	51362 → 443 [ACK] Seq=1
37	5.298	2603:1063:200...	2401:4900:93f...	TCP	90	443 → 54015 [ACK] Seq=1
42	5.331	2401:4900:93f...	2603:1063:220...	TLS...	206	Application Data
43	5.331	2401:4900:93f...	2603:1063:220...	TLS...	113	Application Data

Packet Details:

- Frame 2: Packet, 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface 0
- Ethernet II, Src: 66:c6:c8:52:9a:80 (66:c6:c8:52:9a:80), Dst: Intel_55:61:7b (4c:4d:56:00:00:00)
- Internet Protocol Version 6, Src: 2603:1040:5:3::a, Dst: 2401:4900:93f0:2e53:6c61::
- Transmission Control Protocol, Src Port: 443, Dst Port: 60113, Seq: 1, Ack: 29, Len: 0
- Source Port: 443
- Destination Port: 60113
- [Stream index: 0]
- [Stream Packet Number: 2]
- [Conversation completeness: Incomplete (12)]
- ...0... = RST: Absent
- ...0... = FIN: Absent
- ...1... = Data: Present
- ...1... = ACK: Present
- ...0... = SYN-ACK: Absent
- ...0... = SYN: Absent
- [Completeness Flags: ..DA..]
- [TCP Segment Len: 0]
- Sequence Number: 1 (relative sequence number)
- Sequence Number (raw): 1034168310
- [Next Sequence Number: 1 (relative sequence number)]
- Acknowledgment Number: 29 (relative ack number)
- Acknowledgment number (raw): 2959067361
- 0101... = Header Length: 20 bytes (5)
- Flags: 0x010 (ACK)
- Window: 644
- [Calculated window size: 644]
- [Window size scaling factor: -1 (unknown)]
- Checksum: 0xfe54 [unverified]
- [Checksum Status: Unverified]
- Urgent Pointer: 0
- [Timestamps]
- [SEQ/ACK analysis]
- [Client Contiguous Streams: 1]
- [Server Contiguous Streams: 1]

Raw Data:

```

0000  4c 49 6c 55 61 7b 66 c6  c8 52 9a 80 86 dd 60 0d  LI1Ua{f...R...
0010  c8 e1 50 10 02 84 fe 54  0000
  
```

Transmission... Packets: 10571 · Displayed: 3773 (35.7%) · Dropped: 0 (0.0%) · Profile: Default

1. Packet Header, Options and Flag

The TCP packet header is of TCP header is 20 bytes minimum and can grow up to 60 bytes with options, consisting of,

- Source Port (16 bits):** The port number of the application on the sending device (e.g., a random high port like 51234 on your browser).
- Destination Port (16 bits):** The port number of the service on the receiving device (e.g., Port 80 for HTTP or 443 for HTTPS).
- Sequence Number (32 bits):** This is the "ID card" for each byte of data. It ensures that if packets arrive out of order, the receiver can reassemble them correctly. In a SYN packet, this is the Initial Sequence Number (ISN).
- Acknowledgment Number (32 bits):** Used only if the ACK flag is set. It tells the sender: "I have received everything up to byte X, and I am now expecting byte X+1."
- Data Offset (4 bits):** Specifies the size of the TCP header in 32-bit words. This is necessary because the Options field makes the header length variable.
- Reserved (3 bits):** Always set to 0. Reserved for future use (similar to the Z flag in DNS).
- Flags (9 bits - The "Control" Bits):** These are the most critical bits for managing the connection:

- NS/CWR/ECE (3 bits): Used for Congestion Notification (managing network traffic jams).
- URG (1 bit): Urgent pointer field set to know how important the data is.
- ACK (1 bit): Acknowledgment. Set in almost every packet after the first SYN.
- PSH (1 bit): Push. Tells the receiver to send the data to the app immediately, don't wait for the buffer to fill.
- RST (1 bit): Reset. Forcibly kills the connection due to an error.
- SYN (1 bit): Synchronize. Used to initiate a 3-way handshake.
- FIN (1 bit): Finish. Used to gracefully close a connection.

h) Window Size (16 bits): Used for Flow Control. The receiver uses this to say: "I can only handle X more bytes of data right now—don't send more than that!"

i) Checksum (16 bits): Used for error-checking the header and the data to ensure nothing was corrupted during transit.

2. TCP Packet Flow:

a) During session establishment

Before data can move, the two devices must agree on sequence numbers and capabilities (like Window Size). This is a three-step process:

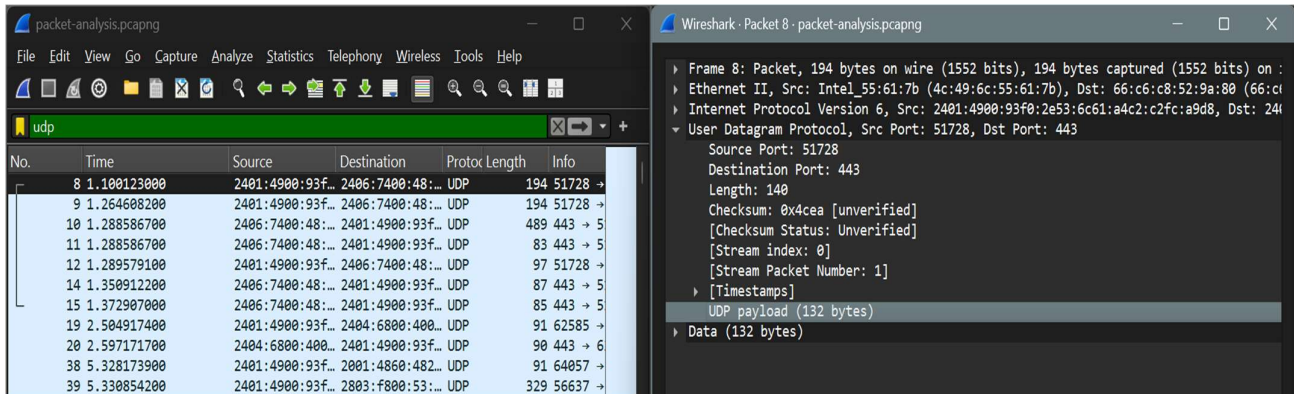
- SYN (Synchronize):** The client sends a packet with the SYN flag set and a random Initial Sequence Number (ISN)
- SYN-ACK (Synchronize-Acknowledgement):** The server responds with both SYN and ACK flags set. It generates its own ISN (e.g., Seq=500) and sets the Acknowledgment number to the client's ISN + 1 (Ack=101).
- ACK:** The client sends a final packet with the ACK flag set. It increments its sequence number (Seq=101) and sets the Acknowledgment to the server's ISN + 1 (Ack=501).

b) During session termination

Closing a connection is more complex because TCP is full-duplex. This means the Client can be finished sending data while the Server still has a few more bytes to "flush" out. Therefore, each side must close its half of the connection independently.

- FIN (Finish by client):** The client sends a packet with FIN flag set.
- ACK (Acknowledgement):** The server sends the ack that it received the FIN packet [CLOSE_WAIT].
- Fin (Finish by server):** The server sends a packet with FIN flag set
- ACK (Acknowledgement):** The client sends a final ACK [TIME_WAIT]

Analyzing UDP Packets



Packet Header, Options and Flags

The User Datagram Protocol header is very small (8 bytes) and has only 4 fields:

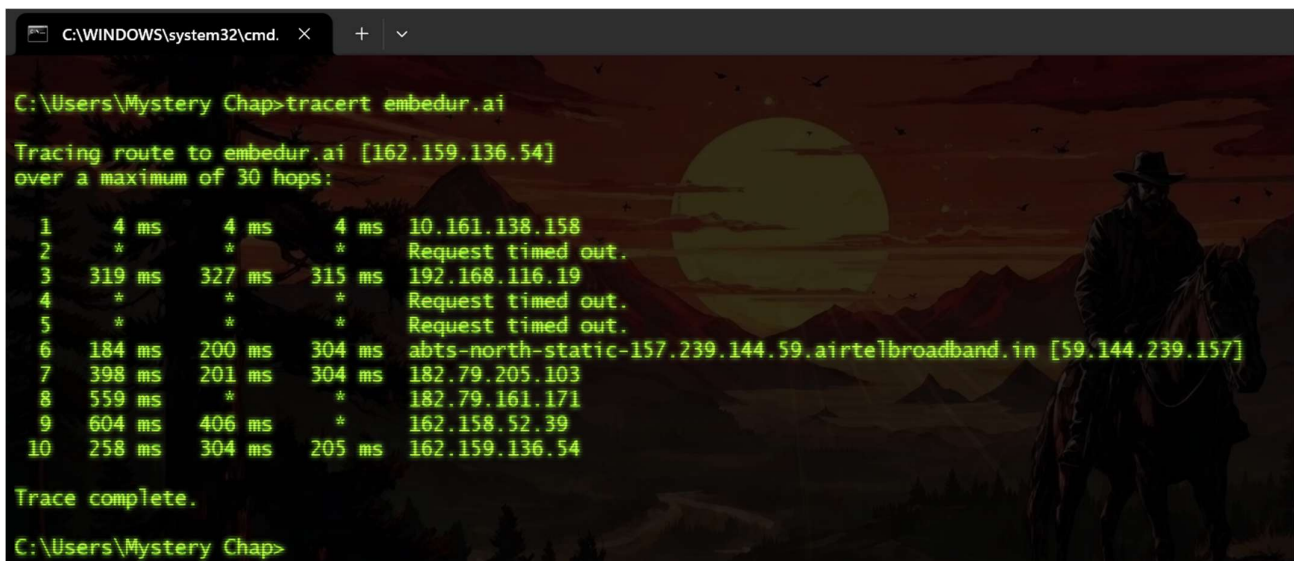
- Source Port (2 bytes) → Sender's port number
- Destination Port (2 bytes) → Receiver's port number
- Length (2 bytes) → Total length of UDP (header + data)
- Checksum (2 bytes) → Error detection (optional in IPv4, mandatory in IPv6)

The UDP packets doesn't have an option field or a flag field as it is designed to be simple and fast with less overhead.

The UDP protocol doesn't have any mechanism to ensure the packet delivery or any flow that needed to be followed except the TCP/IP stack.

3. Explore tracert/traceroute to websites and analyse the parameters in the output and explore different options for traceroute command

a) Using Simple tracert with no flag



- b) Using tracert with -d flag => Prevents IP address resolution speeding up the process.

```
C:\Users\Mystery Chap>tracert -d embedur.ai

Tracing route to embedur.ai [162.159.136.54]
over a maximum of 30 hops:

  1    13 ms    4 ms    4 ms    10.161.138.158
  2   192 ms    *    *    192.168.72.130
  3   226 ms   202 ms   51 ms   192.168.116.19
  4    *    *    *    Request timed out.
  5    *    *    *    Request timed out.
  6    77 ms   47 ms   41 ms   59.144.239.157
  7    93 ms   243 ms  114 ms   182.79.205.103
  8    *   101 ms    *   182.79.161.171
  9    58 ms   45 ms   85 ms   162.158.52.39
 10   315 ms   89 ms  154 ms   162.159.136.54

Trace complete.
```

- c) Enforcing to use IPv4 only

```
C:\Users\Mystery Chap>tracert -d -4 google.com

Tracing route to google.com [142.251.221.174]
over a maximum of 30 hops:

  1     4 ms     3 ms     3 ms    10.161.138.158
  2    *    *    *    Request timed out.
  3   403 ms   362 ms   335 ms   192.168.116.19
  4    *   213 ms   202 ms   192.168.116.49
  5    *    *    *    Request timed out.
  6    51 ms    74 ms    61 ms   122.187.102.81
  7    67 ms    51 ms    61 ms   116.119.68.247
  8    93 ms    66 ms    50 ms   72.14.205.196
  9    70 ms    57 ms    66 ms   142.251.227.211
 10    81 ms    88 ms   191 ms   142.251.55.235
 11   189 ms   100 ms    45 ms   142.251.221.174

Trace complete.
```

- d) Enforcing to use IPv6 only

```
C:\Users\Mystery Chap>tracert -d -6 google.com

Tracing route to google.com [2404:6800:4007:81b::200e]
over a maximum of 30 hops:

  1     6 ms     3 ms     5 ms   2401:4900:93f0:2e53::59
  2   274 ms   303 ms   304 ms   2401:4900:1:a809::3
  3   311 ms   402 ms   203 ms   2401:4900:1:a883::3
  4   316 ms   356 ms   441 ms   2401:4900:1:a882::1
  5    *    *    *    Request timed out.
  6   482 ms   407 ms   614 ms   2404:a800:3a00:300::55
  7    *    *    *    Request timed out.
  8   499 ms   402 ms   612 ms   2404:6800:8201:180::1
  9   356 ms   463 ms   302 ms   2404:6800:8201:180::1
 10   513 ms   611 ms   304 ms   2404:6800:4007:81b::200e

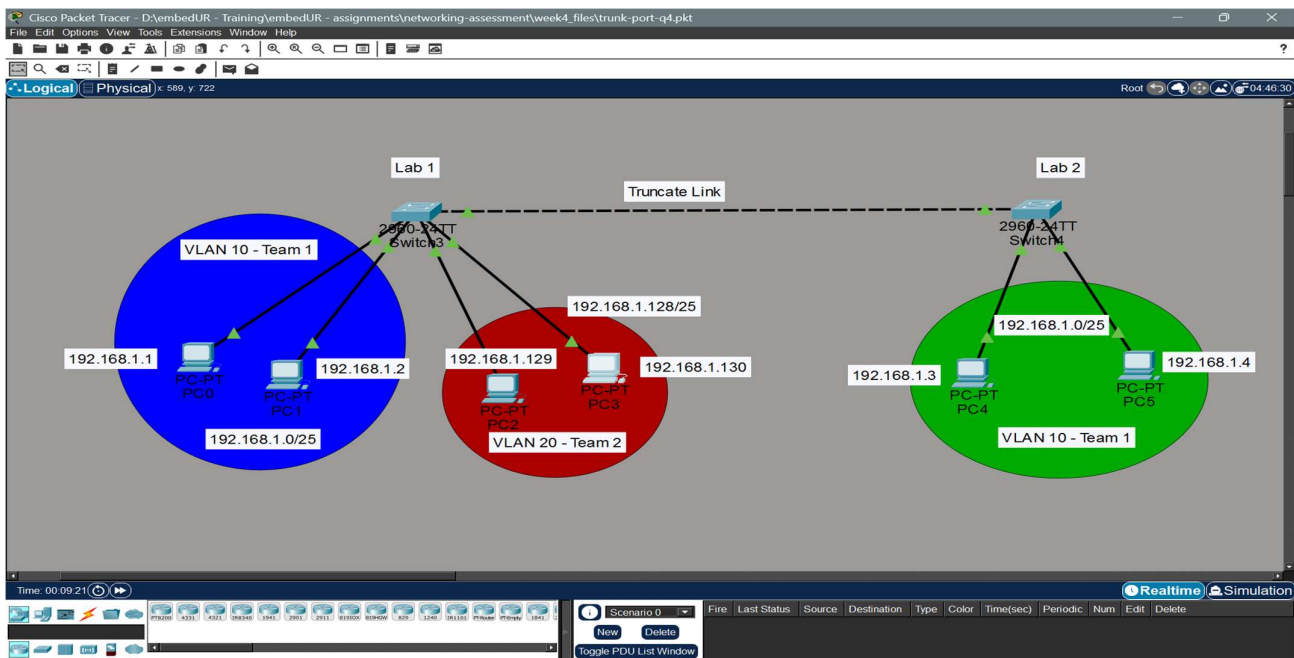
Trace complete.
```

Similar to this there are many other flags. Such as specify max hops, trace RTT and timeout. The common symbols in tracer are,

- a) * * * (Request Time Out): The packet did not return within the timeout period, often due to firewalls or high load.
- b) !A: Administratively prohibited (blocked).
- c) !N: Network unreachable.
- d) !P: Protocol unreachable.

5) Set up a trunk port between switches and try ping between different VLANs

a) For experiment I have created the following topology,



b) Create VLANs in the respective switches.

Switch 3

```
Switch>en
Switch#conf t
Enter configuration commands, one per line. End with CNTL/Z.
Switch(config)#vlan 10
Switch(config-vlan)#name Team1
Switch(config-vlan)#exit
Switch(config)#vlan 20
Switch(config-vlan)#name Team2
Switch(config-vlan)#do show vlan brief
```

VLAN	Name	Status	Ports
1	default	active	Fa0/1, Fa0/2, Fa0/3, Fa0/4 Fa0/5, Fa0/6, Fa0/7, Fa0/8 Fa0/9, Fa0/10, Fa0/11, Fa0/12 Fa0/13, Fa0/14, Fa0/15, Fa0/16 Fa0/17, Fa0/18, Fa0/19, Fa0/20 Fa0/21, Fa0/22, Fa0/23, Fa0/24 Gig0/1, Gig0/2
10	Team1	active	
20	Team2	active	
1002	fddi-default	active	
1003	token-ring-default	active	
1004	fddinet-default	active	
1005	trnet-default	active	

Switch 4:

```
Switch>en
Switch#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
Switch(config)#vlan 10
Switch(config-vlan)#name team1
Switch(config-vlan)#do show vlan brief
```

VLAN	Name	Status	Ports
1	default	active	Fa0/1, Fa0/2, Fa0/3, Fa0/4 Fa0/5, Fa0/6, Fa0/7, Fa0/8 Fa0/9, Fa0/10, Fa0/11, Fa0/12 Fa0/13, Fa0/14, Fa0/15, Fa0/16 Fa0/17, Fa0/18, Fa0/19, Fa0/20 Fa0/21, Fa0/22, Fa0/23, Fa0/24 Gig0/1, Gig0/2
10	team1	active	
1002	fddi-default	active	
1003	token-ring-default	active	
1004	fddinet-default	active	
1005	trnet-default	active	

```
Switch(config-vlan)#
```

c) Configure the VLAN to the ports (access VLAN)

Switch 3:

```
Switch(config)#
Switch(config)#int range fa
Switch(config)#int range fastEthernet 0/1-2
Switch(config-if-range)#switchport mode a
Switch(config-if-range)#switchport mode access
Switch(config-if-range)#switchport access vlan 10
Switch(config-if-range)#exit
Switch(config)#int range fastEthernet 0/3-4
Switch(config-if-range)#switchport access vlan 20
Switch(config-if-range)#switchport mode access
Switch(config-if-range)#exit
Switch(config)#do show vlan br
Switch(config)#do show vlan bri
Switch(config)#do show vlan brief
```

VLAN	Name	Status	Ports
1	default	active	Fa0/5, Fa0/6, Fa0/7, Fa0/8 Fa0/9, Fa0/10, Fa0/11, Fa0/12 Fa0/13, Fa0/14, Fa0/15, Fa0/16 Fa0/17, Fa0/18, Fa0/19, Fa0/20 Fa0/21, Fa0/22, Fa0/23, Fa0/24 Gig0/1, Gig0/2
10	Team1	active	Fa0/1, Fa0/2
20	Team2	active	Fa0/3, Fa0/4
1002	fddi-default	active	
1003	token-ring-default	active	
1004	fddinet-default	active	
1005	trnet-default	active	

```
Switch(config)#
```

Switch 4:

```
Switch>en
Switch#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
Switch(config)#int con
Switch(config)#int conf
Switch(config)#int r
Switch(config)#int range fa
Switch(config)#int range fastEthernet 0/1-2
Switch(config-if-range)#switchport mode access
Switch(config-if-range)#switchport mode access
Switch(config-if-range)#do show vlan brief
```

VLAN	Name	Status	Ports
1	default	active	Fa0/3, Fa0/4, Fa0/5, Fa0/6 Fa0/7, Fa0/8, Fa0/9, Fa0/10 Fa0/11, Fa0/12, Fa0/13, Fa0/14 Fa0/15, Fa0/16, Fa0/17, Fa0/18 Fa0/19, Fa0/20, Fa0/21, Fa0/22 Fa0/23, Fa0/24, Gig0/1, Gig0/2 Fa0/1, Fa0/2
10	team1	active	
1002	fddi-default	active	
1003	token-ring-default	active	
1004	fddinet-default	active	
1005	trnet-default	active	

```
Switch(config-if-range)#
```


d) Now enable truncate port on the switches

Switch 3:

```
Switch>en
Switch#conf t
Enter configuration commands, one per line. End with CNTL/Z.
Switch(config)#int f
Switch(config)#int fastEthernet 0/24
Switch(config-if)#switchpo
Switch(config-if)#switchport mode tr
Switch(config-if)#switchport mode trunk

Switch(config-if)#
%LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet0/24, changed state to
down

%LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet0/24, changed state to
up

Switch(config-if)#switport trunk native vlan 1
^
% Invalid input detected at '^' marker.

Switch(config-if)#switchport trunk native vlan 1
Switch(config-if)#switchport trunk allowe
Switch(config-if)#switchport trunk allowed vlan 10
Switch(config-if)#do copy runn
Switch(config-if)#do copy running-config starpup-config
copy running-config starpup-config
^
% Invalid input detected at '^' marker.

Switch(config-if)#do copy running-config startup-cnfig
Destination filename [startup-config]?
Building configuration...
[OK]
Switch(config-if)#
```

Switch 4:

```
Switch(config)#int fa
Switch(config)#int fa0/24
Switch(config-if)#switchport mode trunk
Switch(config-if)#switchport trunk n
Switch(config-if)#switchport trunk na
Switch(config-if)#switchport trunk native vl
Switch(config-if)#switchport trunk native vlan 1
Switch(config-if)#switchport trunk ?
    allowed Set allowed VLAN characteristics when interface is in trunking mode
    native Set trunking native characteristics when interface is in trunking
    mode
Switch(config-if)#switchport trunk allowed vlan 10
Switch(config-if)#do copy running-config startup-config
Destination filename [startup-config]?
Building configuration...
[OK]
Switch(config-if)#show interfaces trunk
^
% Invalid input detected at '^' marker.

Switch(config-if)#do show interfaces trunk
Port      Mode      Encapsulation      Status      Native vlan
Fa0/24    on        802.1q              trunking    1

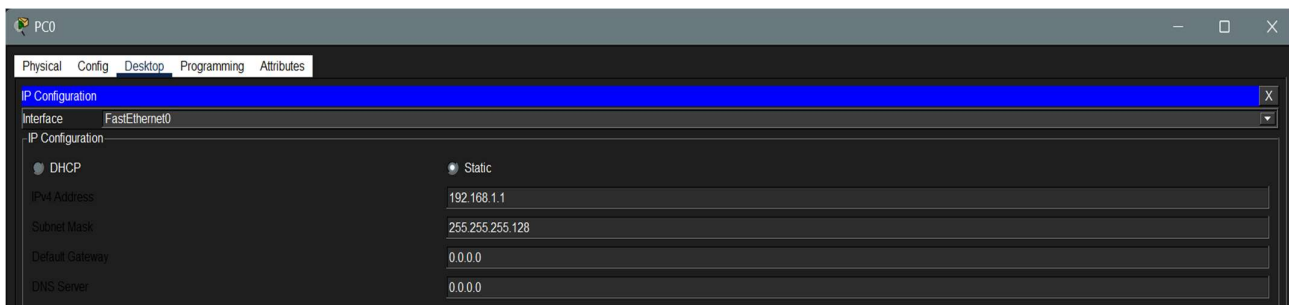
Port      Vlans allowed on trunk
Fa0/24    10

Port      Vlans allowed and active in management domain
Fa0/24    10

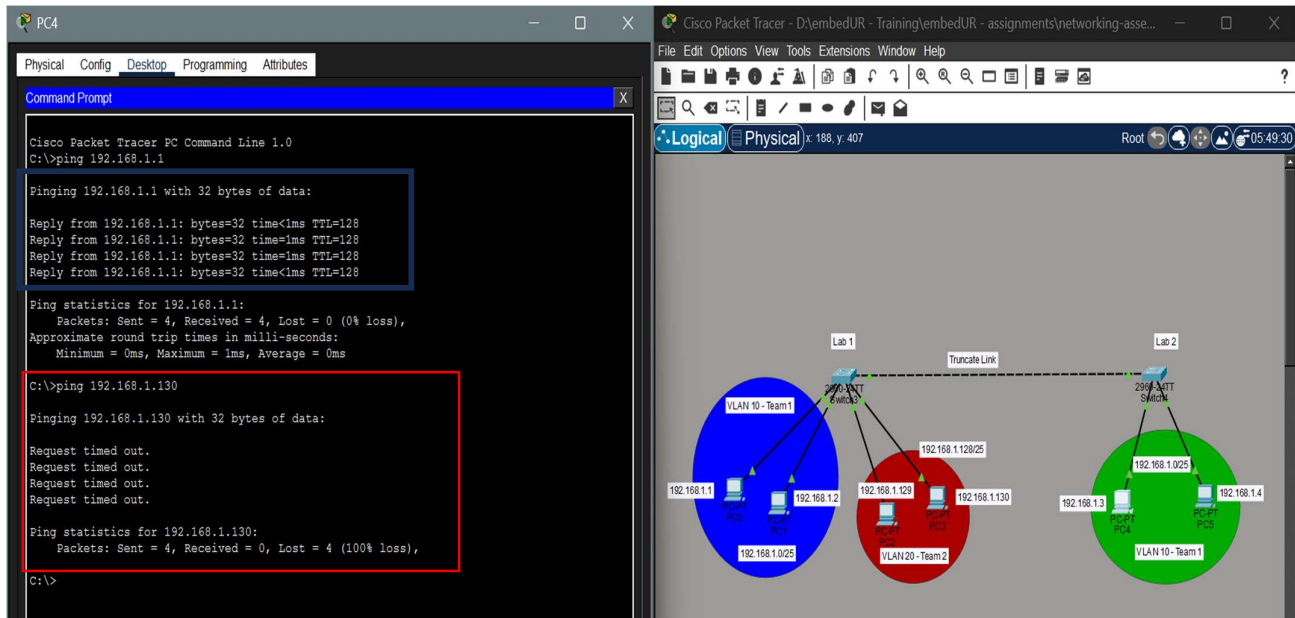
Port      Vlans in spanning tree forwarding state and not pruned
Fa0/24    10

Switch(config-if)#|
```

e) Configure the IP address in each system.



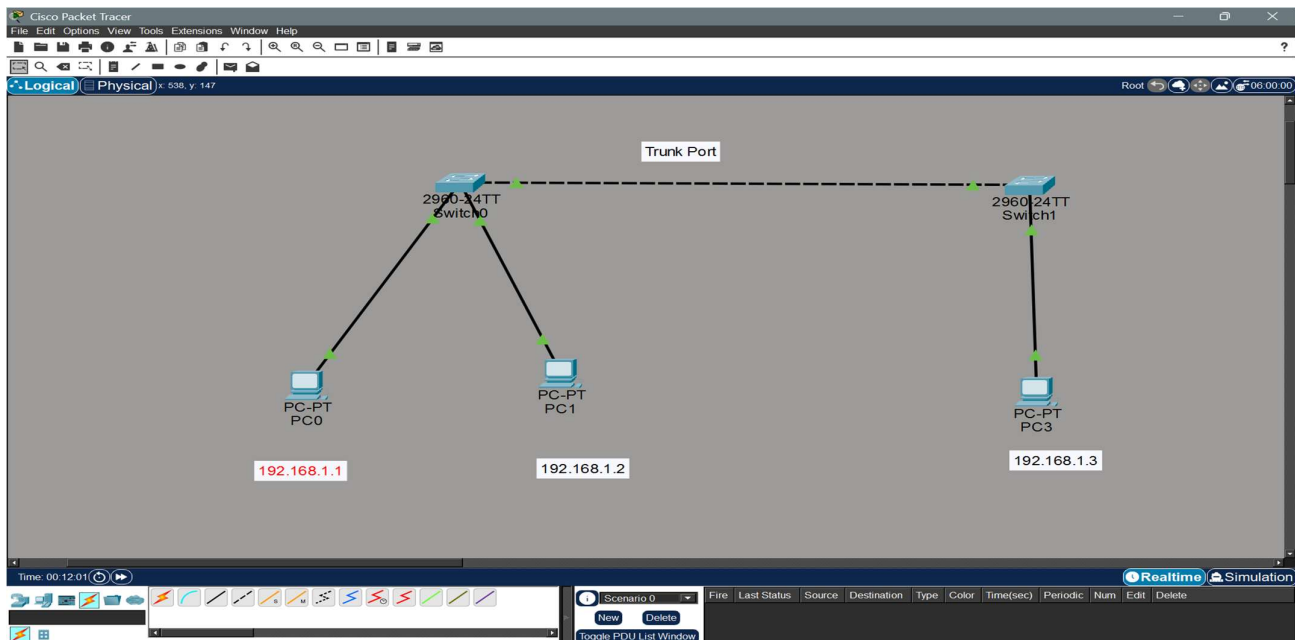
f) Now let us ping and test whether the system allows the traffic from VLAN 10 in Switch 3 to VLAN 10 in switch 4.



We can see that traffic from VLAN 10 – Switch 3 reaches the traffic from VLAN 10 – Switch 4. But the VLAN 20 didn't get any traffic from VLAN 10 from switch 4.

6) Change the native VLAN on the trunk port. Test the VLAN mismatch and troubleshoot.

a) I have used the following Topology for this experiment.



b) Configure the Trunk Port

Switch 0:

```

Switch>
Switch>en
Switch#conf t
Enter configuration commands, one per line. End with CNTL/Z.
Switch(config)#int fa0/24
Switch(config-if)#?
  authentication      Auth Manager Interface Configuration Commands
  cdp                 Global CDP configuration subcommands
  channel-group       Etherchannel/port bundling configuration
  channel-protocol    Select the channel protocol (LACP, PAgP)
  delay              Specify interface throughput delay
  description         Interface specific description
  dot1x              Interface Config Commands for IEEE 802.1X
  duplex             Configure duplex operation.
  exit               Exit from interface configuration mode
  ip                 Interface Internet Protocol config commands
  lldp               LLDP interface subcommands
  mdix               Set Media Dependent Interface with Crossover
  mls                mls interface commands
  no                 Negate a command or set its defaults
  shutdown           Shutdown the selected interface
  spanning-tree       Spanning Tree Subsystem
  speed              Configure speed operation.
  storm-control       storm configuration
  switchport         Set switching mode characteristics
  tx-ring-limit       Configure PA level transmit ring limit
Switch(config-if)#switchport mode trunk

Switch(config-if)#
%LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet0/24, changed state to down

%LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet0/24, changed state to up

Switch(config-if)#switchport trunk native
Switch(config-if)#switchport trunk native vlan 1

```

Switch#show vlan brief

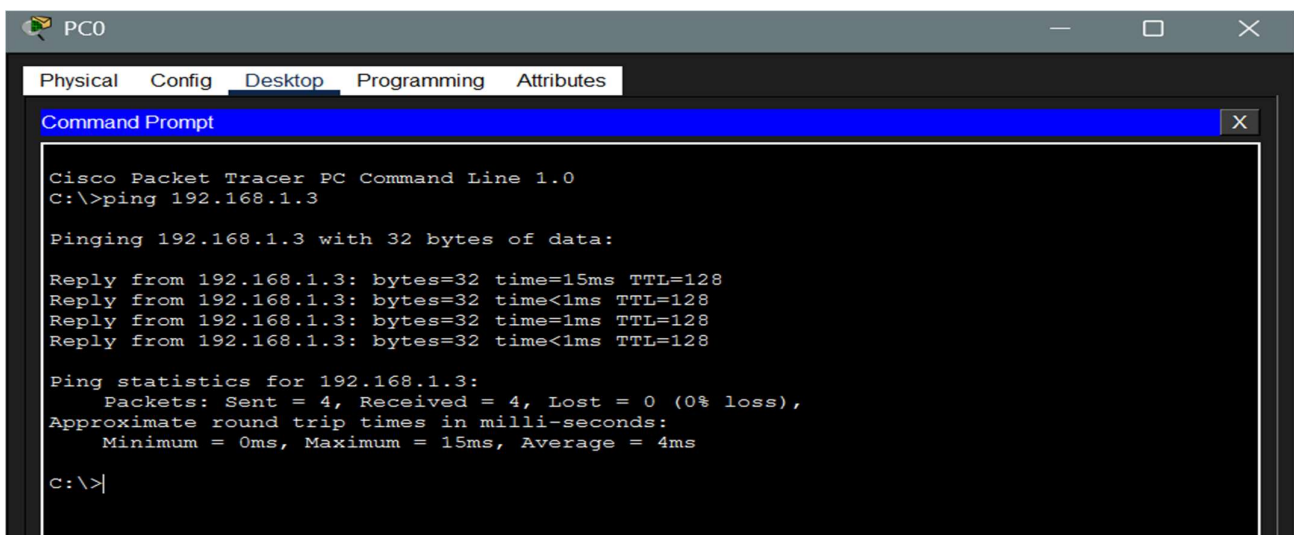
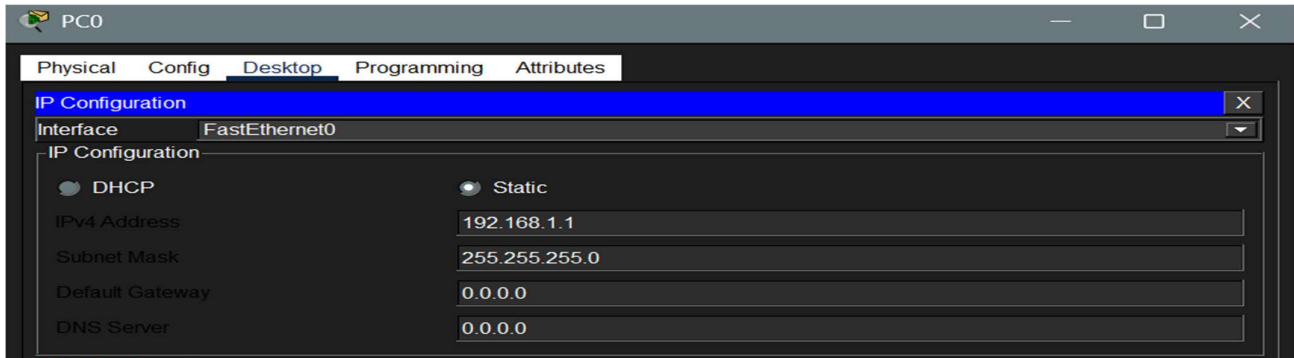
VLAN	Name	Status	Ports
1	default	active	Fa0/1, Fa0/2, Fa0/3, Fa0/4 Fa0/5, Fa0/6, Fa0/7, Fa0/8 Fa0/9, Fa0/10, Fa0/11, Fa0/12 Fa0/13, Fa0/14, Fa0/15, Fa0/16 Fa0/17, Fa0/18, Fa0/19, Fa0/20 Fa0/21, Fa0/22, Fa0/23, Gig0/1 Gig0/2
1002	fdi-default	active	
1003	token-ring-default	active	
1004	fdinet-default	active	
1005	trnet-default	active	

Port	Name	Status	Vlan	Duplex	Speed	Type
Fa0/1		connected	1	a-full	a-100	10/100BaseTX
Fa0/2		connected	1	a-full	a-100	10/100BaseTX
Fa0/3		notconnect	1	auto	auto	10/100BaseTX
Fa0/4		notconnect	1	auto	auto	10/100BaseTX
Fa0/5		notconnect	1	auto	auto	10/100BaseTX
Fa0/6		notconnect	1	auto	auto	10/100BaseTX
Fa0/7		notconnect	1	auto	auto	10/100BaseTX
Fa0/8		notconnect	1	auto	auto	10/100BaseTX
Fa0/9		notconnect	1	auto	auto	10/100BaseTX
Fa0/10		notconnect	1	auto	auto	10/100BaseTX
Fa0/11		notconnect	1	auto	auto	10/100BaseTX
Fa0/12		notconnect	1	auto	auto	10/100BaseTX
Fa0/13		notconnect	1	auto	auto	10/100BaseTX
Fa0/14		notconnect	1	auto	auto	10/100BaseTX
Fa0/15		notconnect	1	auto	auto	10/100BaseTX
Fa0/16		notconnect	1	auto	auto	10/100BaseTX
Fa0/17		notconnect	1	auto	auto	10/100BaseTX
Fa0/18		notconnect	1	auto	auto	10/100BaseTX
Fa0/19		notconnect	1	auto	auto	10/100BaseTX
Fa0/20		notconnect	1	auto	auto	10/100BaseTX
Fa0/21		notconnect	1	auto	auto	10/100BaseTX
Fa0/22		notconnect	1	auto	auto	10/100BaseTX
Fa0/23		notconnect	1	auto	auto	10/100BaseTX
Fa0/24		connected	trunk	a-full	a-100	10/100BaseTX
Gig0/1		notconnect	1	auto	auto	10/100/1000BaseTX
Gig0/2		notconnect	1	auto	auto	10/100/1000BaseTX

Switch#

Similarly configure in switch 1 also

c) Now let us configure the IP to each system and check it works on Native VLAN 1 by default.



Ping works perfectly before changing the native VLAN 1.

e) No let us change the Native VLAN from 1 to another VLAN (VLAN 2 in my experiment)

Switch 0:

```
Switch#conf t
Enter configuration commands, one per line. End with CNTL/Z.
Switch(config)#vlan 2
Switch(config-vlan)#name sample
Switch(config-vlan)#exit
Switch(config)#int fa
Switch(config)#int fastEthernet 0/24
Switch(config-if)#switchport trunk n
Switch(config-if)#switchport trunk native vl
Switch(config-if)#switchport trunk native vlan 2
Switch(config-if)%%SPANTREE-2-RECV_PVID_ERR: Received BPDU with inconsistent peer vlan
id 1 on FastEthernet0/24 VLAN2.

%SPANTREE-2-BLOCK_PVID_LOCAL: Blocking FastEthernet0/24 on VLAN0002. Inconsistent local
vlan.

Switch(config-if)#
%CDP-4-NATIVE_VLAN_MISMATCH: Native VLAN mismatch discovered on FastEthernet0/24 (2),
with Switch FastEthernet0/24 (1).

Switch(config-if)#exit
Switch(config)#|
```

As soon as configured, we get the following warning, by which itself we can say there is a mismatch.

f) Now let us ping the Switch 1 system with this new configuration. This shows the packet didn't transfer due to mismatch.

```

PCO
Physical Config Desktop Programming Attributes
Command Prompt
Cisco Packet Tracer PC Command Line 1.0
C:\>ping 192.168.1.3

Pinging 192.168.1.3 with 32 bytes of data:

Reply from 192.168.1.3: bytes=32 time=15ms TTL=128
Reply from 192.168.1.3: bytes=32 time<1ms TTL=128
Reply from 192.168.1.3: bytes=32 time=1ms TTL=128
Reply from 192.168.1.3: bytes=32 time<1ms TTL=128

Ping statistics for 192.168.1.3:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 15ms, Average = 4ms

C:\>ping 192.168.1.3

Pinging 192.168.1.3 with 32 bytes of data:

Request timed out.
Request timed out.
Request timed out.
Request timed out.

Ping statistics for 192.168.1.3:
    Packets: Sent = 4, Received = 0, Lost = 4 (100% loss),
  
```

g) To troubleshoot the mismatch, we

i) Look for error message in console

```

Switch(config)#
%CDP-4-NATIVE_VLAN_MISMATCH: Native VLAN mismatch discovered on FastEthernet0/24 (2),
with Switch FastEthernet0/24 (1).

%CDP-4-NATIVE_VLAN_MISMATCH: Native VLAN mismatch discovered on FastEthernet0/24 (2),
with Switch FastEthernet0/24 (1).

%CDP-4-NATIVE_VLAN_MISMATCH: Native VLAN mismatch discovered on FastEthernet0/24 (2),
with Switch FastEthernet0/24 (1).

%CDP-4-NATIVE_VLAN_MISMATCH: Native VLAN mismatch discovered on FastEthernet0/24 (2),
with Switch FastEthernet0/24 (1).

%CDP-4-NATIVE_VLAN_MISMATCH: Native VLAN mismatch discovered on FastEthernet0/24 (2),
with Switch FastEthernet0/24 (1).

%CDP-4-NATIVE_VLAN_MISMATCH: Native VLAN mismatch discovered on FastEthernet0/24 (2),
with Switch FastEthernet0/24 (1).
  
```

ii) Run show interface trunk to see conf detail in both switch

```

Switch#show interface trunk
Port    Mode      Encapsulation  Status        Native vlan
Fa0/24   on        802.1q         trunking      2

Port      Vlans allowed on trunk
Fa0/24    1-1005

Port      Vlans allowed and active in management domain
Fa0/24    1,2

Port      Vlans in spanning tree forwarding state and not pruned
Fa0/24    none

Switch>show interface trunk
Port    Mode      Encapsulation  Status        Native vlan
Fa0/24   on        802.1q         trunking      1

Port      Vlans allowed on trunk
Fa0/24    1-1005

Port      Vlans allowed and active in management domain
Fa0/24    1

Port      Vlans in spanning tree forwarding state and not pruned
Fa0/24    none
  
```

We can see there is a mismatch in VLAN IDs. Now we can correct it easily.

In switch 0, go to trunk interface, run **switchport trunk native vlan 1**

```
Switch(config-if)#switchport tr
Switch(config-if)#switchport trunk n|
Switch(config-if)#switchport trunk native
%CDP-4-NATIVE_VLAN_MISMATCH: Native VLAN mismatch discovered on FastEthernet0/24 (2),
with Switch FastEthernet0/24 (1).

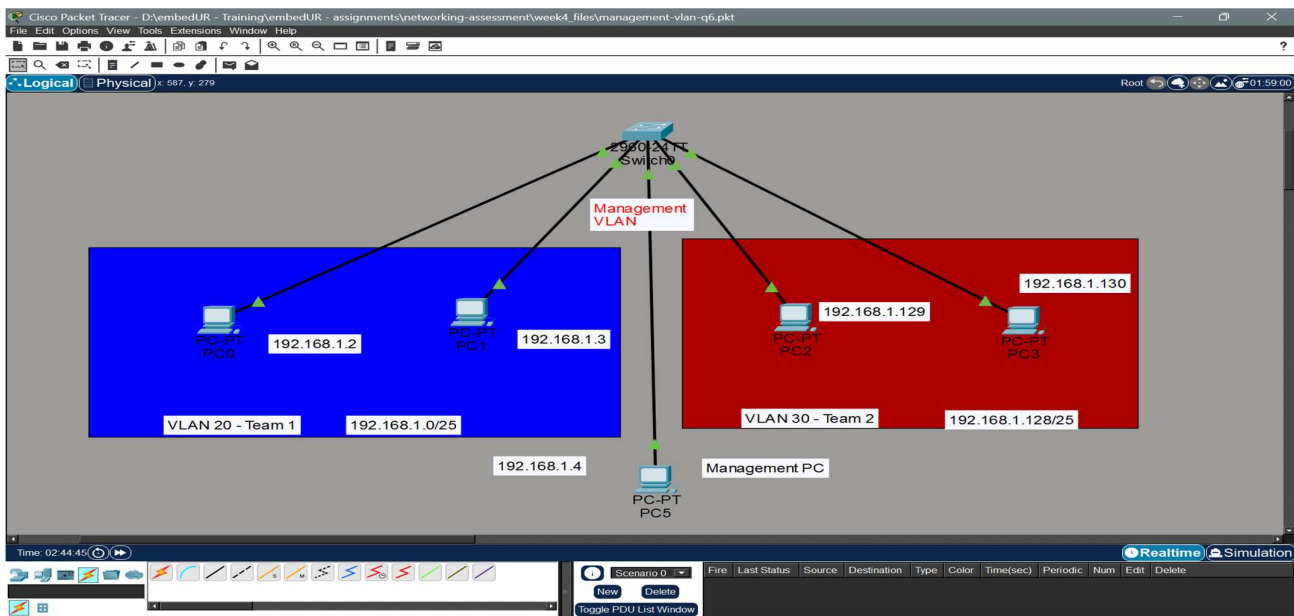
Switch(config-if)#switchport trunk native vlan re
Switch(config-if)#switchport trunk native vlan ?
<1-4094> VLAN ID of the native VLAN when this port is in trunking mode
Switch(config-if)#switchport trunk native vlan 1
Switch(config-if)%%SPANTREE-2-UNBLOCK_CONSIST_PORT: Unblocking FastEthernet0/24 on
VLAN0001. Port consistency restored.

%SPANTREE-2-UNBLOCK_CONSIST_PORT: Unblocking FastEthernet0/24 on VLAN0002. Port
consistency restored.
```

Now the connection is restored successfully.

7. Configure a management VLAN and assign an IP address for remote access. Test SSH or Telnet access to switch

I have used the below topology for the experiment



a) Create the VLANs in the switch 0

```
Switch>enable
Switch#conf t
Enter configuration commands, one per line. End with CNTL/Z.
Switch(config)#vlan 10
Switch(config-vlan)#exit
Switch(config)#vlan 20
Switch(config-vlan)#exit
Switch(config)#vlan 30
Switch(config-vlan)#exit
Switch(config)#
```


b) Assign the systems connected to the respective interfaces to the respective vlan

```
Switch(config)#int r
Switch(config)#int range fa
Switch(config)#int range fastEthernet 0/1-2
Switch(config-if-range)#switchport mode access
Switch(config-if-range)#switchport access vlan 20
Switch(config-if-range)#exit
Switch(config)#int range fastEthernet 0/3-4
Switch(config-if-range)#switchport mode access
Switch(config-if-range)#switchport access vlan 30
Switch(config-if-range)#exit
Switch(config)#do show vlan brief
```

VLAN	Name	Status	Ports
1	default	active	Fa0/5, Fa0/6, Fa0/7, Fa0/8 Fa0/9, Fa0/10, Fa0/11, Fa0/12 Fa0/13, Fa0/14, Fa0/15, Fa0/16 Fa0/17, Fa0/18, Fa0/19, Fa0/20 Fa0/21, Fa0/22, Fa0/23, Fa0/24 Gig0/1, Gig0/2
10	VLAN0010	active	
20	VLAN0020	active	Fa0/1, Fa0/2
30	VLAN0030	active	Fa0/3, Fa0/4
1002	fddi-default	active	
1003	token-ring-default	active	
1004	fddinet-default	active	
1005	trnet-default	active	

```
Switch(config)#
```

c) Configure management VLAN with IP using SVI

```
Switch(config)#interface vlan 10
Switch(config-if)#
%LINK-5-CHANGED: Interface Vlan10, changed state to up

Switch(config-if)#ip address 192.168.1.1 255.255.255.0
^
% Invalid input detected at '^' marker.

Switch(config-if)#ip address 192.168.1.1 255.255.255.0
Switch(config-if)#no shutdown
Switch(config-if)#
```

d) Assign an interface for Management VLAN 10 else it will be down.

```
Switch(config)#int fastEthernet 0/5
Switch(config-if)#switchport mode access
^
% Invalid input detected at '^' marker.

Switch(config-if)#switchport mode access
Switch(config-if)#switchport access vlan 10
Switch(config-if)#do show vla brief
```

VLAN	Name	Status	Ports
1	default	active	Fa0/6, Fa0/7, Fa0/8, Fa0/9 Fa0/10, Fa0/11, Fa0/12, Fa0/13 Fa0/14, Fa0/15, Fa0/16, Fa0/17 Fa0/18, Fa0/19, Fa0/20, Fa0/21 Fa0/22, Fa0/23, Fa0/24, Gig0/1 Gig0/2
10	VLAN0010	active	Fa0/5
20	VLAN0020	active	Fa0/1, Fa0/2
30	VLAN0030	active	Fa0/3, Fa0/4
1002	fddi-default	active	
1003	token-ring-default	active	
1004	fddinet-default	active	
1005	trnet-default	active	

```
Switch(config-if)#
```

e) Configure SSH / TELNET service in the switch

```

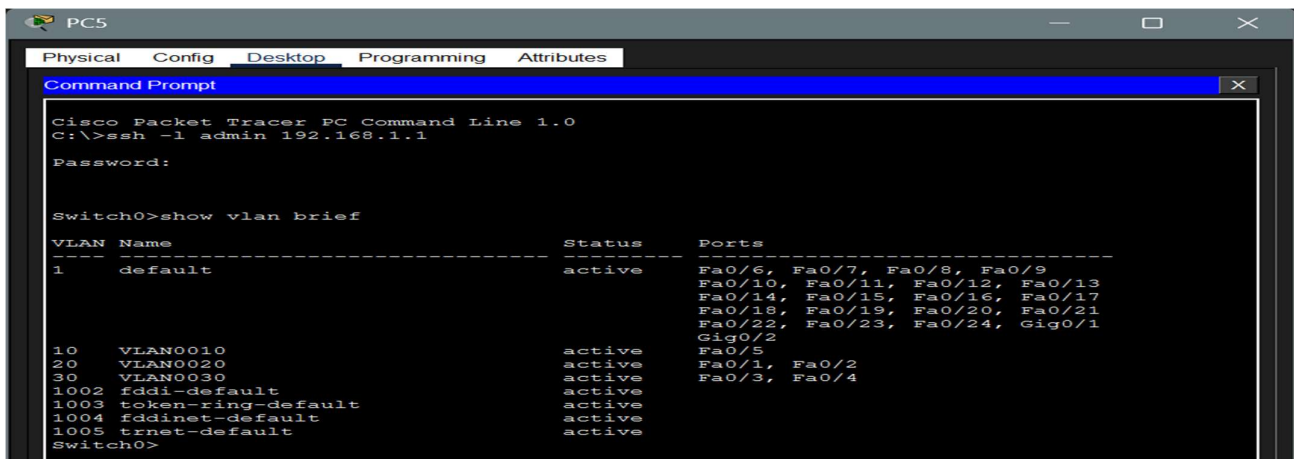
Switch#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
Switch(config)#hostname ?
WORD    This system's network name
Switch(config)#hostname Switch0
Switch0(config)#ip domain
Switch0(config)#ip domain-name lab.local
Switch0(config)#crypto key generate rsa
The name for the keys will be: Switch0.lab.local
Choose the size of the key modulus in the range of 360 to 4096 for your
General Purpose Keys. Choosing a key modulus greater than 512 may take
a few minutes.

How many bits in the modulus [512]:
% Generating 512 bit RSA keys, keys will be non-exportable...[OK]

Switch0(config)#username admin secret admin9486
*Mar 1 2:36:0.301: RSA key size needs to be at least 768 bits for ssh version 2
*Mar 1 2:36:0.303: %SSH-5-ENABLED: SSH 1.5 has been enabled
Switch0(config)#line vty 0 4
Switch0(config-line)#login local
Switch0(config-line)#transport input ssh
Switch0(config-line)#exit
Switch0(config)#

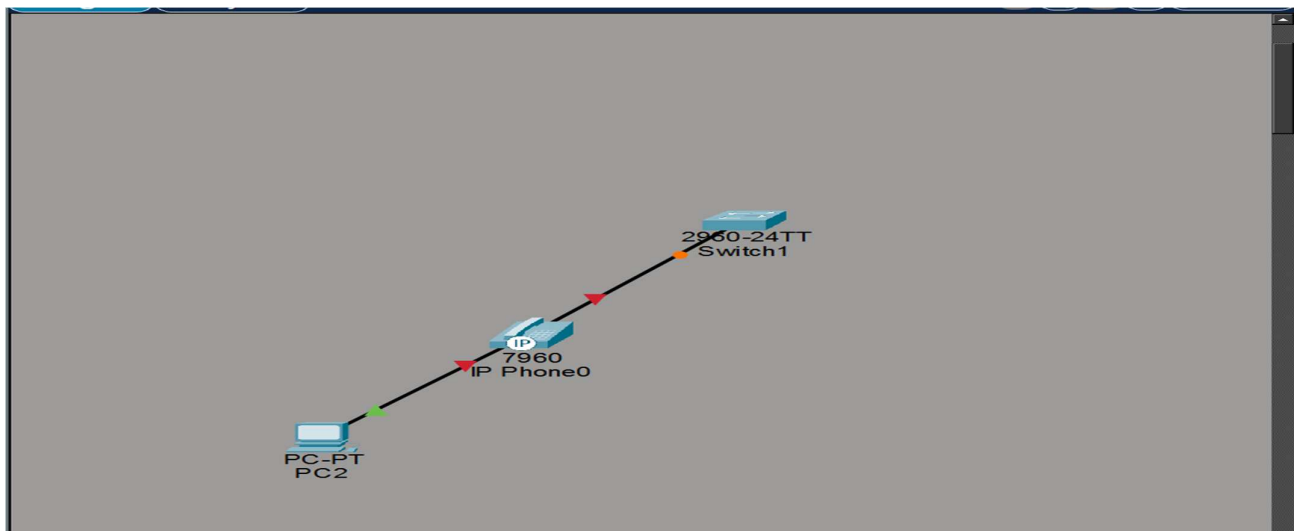
```

f) Now connect a PC to the port assigned for the management VLAN assign it an IP and use it access the management VLAN



8. You have a CISCO switch and a VoIP phone that need to be placed in a voice VLAN (VLAN 20). The data for the PC should remain in a separate VLAN (VLAN 10). Configure the switch port to support both voice and data traffic

a) Create a topology with a CISCO VoIP phone (for voice VLAN) and PC (for data VLAN) through switch.



b) Now create and configure the VLAN for Voice and data.


Switch1

Physical Config CLI Attributes

```

Switch>en
Switch#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
Switch(config)#vlan 10
Switch(config-vlan)#name data-vlan
Switch(config-vlan)#exit
Switch(config)#vlan 20
Switch(config-vlan)#name voice-vlan
Switch(config-vlan)#exit
Switch(config)#interface fa0/1
Switch(config-if)#switchport mode access
Switch(config-if)#switchport access vlan 10
Switch(config-if)#switchport access vlan 20
Switch(config-if)#spanning-tree ?
    bpduguard    Don't send or receive BPDUs on this interface
    bpduguard    Don't accept BPDUs on this interface
    cost         Change an interface's spanning tree port path cost
    guard        Change an interface's spanning tree guard mode
    link-type    Specify a link type for spanning tree protocol use
    portfast     Enable an interface to move directly to forwarding on link up
    vlan         VLAN Switch Spanning Tree
Switch(config-if)#spanning-tree port
Switch(config-if)#spanning-tree portfast
%Warning: portfast should only be enabled on ports connected to a single
host. Connecting hubs, concentrators, switches, bridges, etc... to this
interface when portfast is enabled, can cause temporary bridging loops.
Use with CAUTION

%Portfast has been configured on FastEthernet0/1 but will only
have effect when the interface is in a non-trunking mode.
Switch(config-if)#show vlan brief
      ^
% Invalid input detected at '^' marker.

Switch(config-if)#do show vlan brief

```

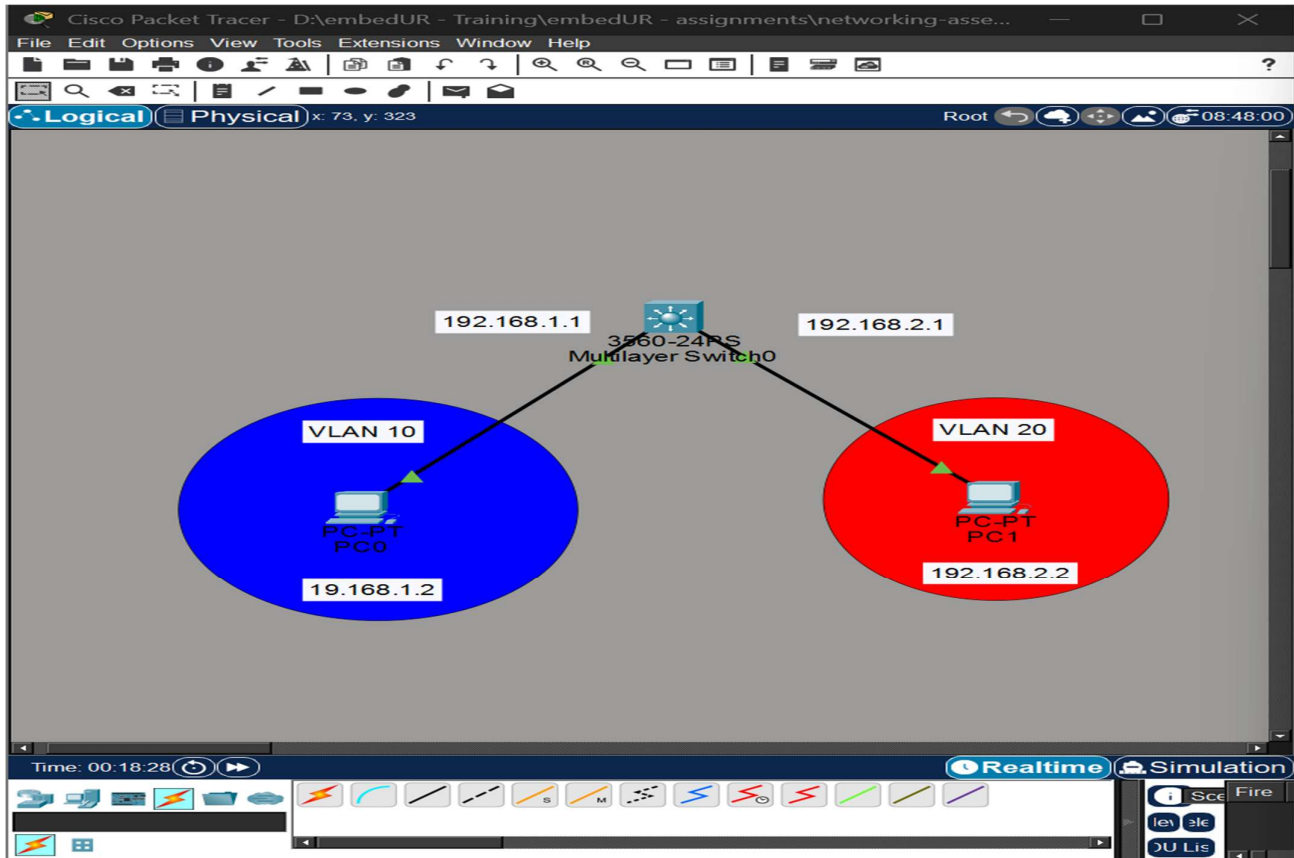
VLAN	Name	Status	Ports
1	default	active	Fa0/2, Fa0/3, Fa0/4, Fa0/5 Fa0/6, Fa0/7, Fa0/8, Fa0/9 Fa0/10, Fa0/11, Fa0/12, Fa0/13 Fa0/14, Fa0/15, Fa0/16, Fa0/17 Fa0/18, Fa0/19, Fa0/20, Fa0/21 Fa0/22, Fa0/23, Fa0/24, Gig0/1 Gig0/2
10	data-vlan	active	
20	voice-vlan	active	Fa0/1
1002	fddi-default	active	
1003	token-ring-default	active	
1004	fddinet-default	active	
1005	trnet-default	active	


Top

9) You configured VLAN 10 and 20 on your switch and assigned ports to each VLAN, however, devices in VLAN 10 can't communicate with devices in VLAN 20. Troubleshoot the issue.

The issue can be solved with Intern-VLAN routing using VLAN's SVI

a) Create a topology as follow



b) Now create VLAN interfaces and assign IP to them.

```
Switch>en
Switch#conf
Configuring from terminal, memory, or network [terminal]?
Enter configuration commands, one per line. End with CNTL/Z.
Switch(config)#vlan 10
Switch(config-vlan)#exit
Switch(config)#vlan 20
Switch(config-vlan)#int fa0/1
Switch(config-if)#switchport mode access
Switch(config-if)#switchport access fa
Switch(config-if)#switchport access v
Switch(config-if)#switchport access vlan 10
Switch(config-if)#ip add
Switch(config-if)#ip addr
Switch(config-if)#exit
Switch(config)#int fa0/2
Switch(config-if)#switchport mode access
Switch(config-if)#switchport access vlan 20
Switch(config-if)#exit
Switch(config)#
```

```

Switch(config)#int vlan 10
Switch(config-if)#
%LINK-5-CHANGED: Interface Vlan10, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface Vlan10, changed state to up

Switch(config-if)#ip address 192.168.1.1 netmask 255.255.255.0
^
% Invalid input detected at '^' marker.

Switch(config-if)#ip address 192.168.1.1 255.255.255.0
Switch(config-if)#no shutdown
Switch(config-if)#
Switch(config-if)#exit
Switch(config)#int vlan 20
Switch(config-if)#
%LINK-5-CHANGED: Interface Vlan20, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface Vlan20, changed state to up

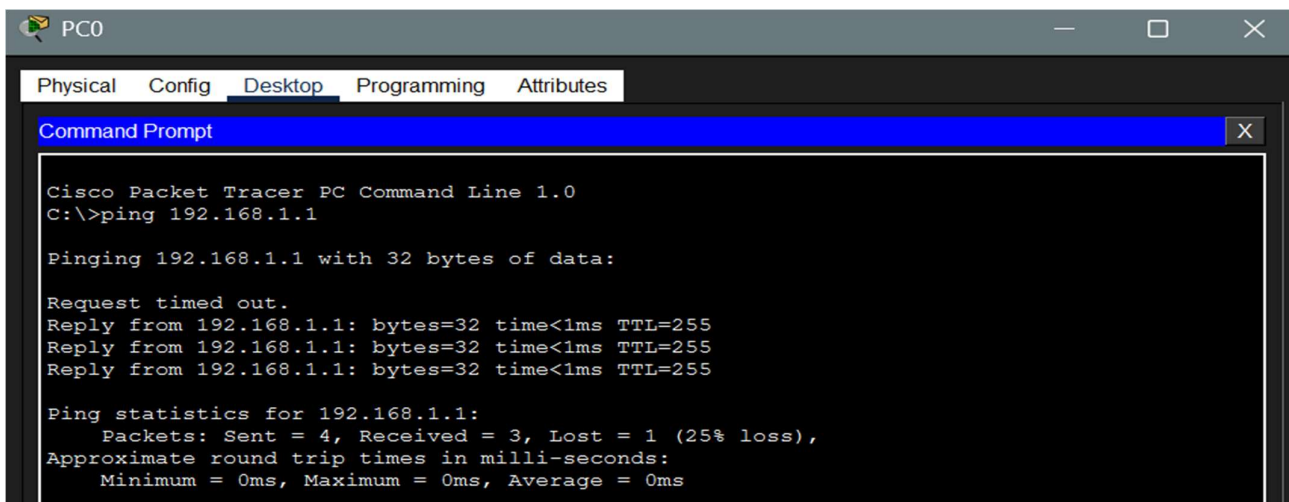
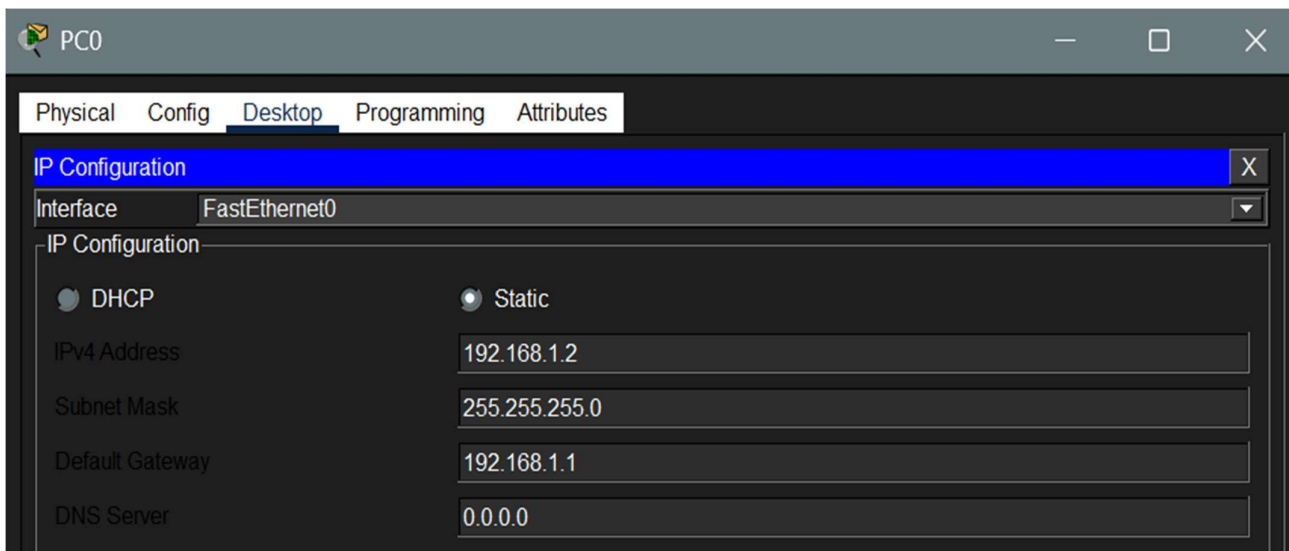
Switch(config-if)#ip address 192.168.2.1 255.255.255.0
Switch(config-if)#no shutdown
Switch(config-if)#exit
Switch(config)#end
Switch#
%SYS-5-CONFIG_I: Configured from console by console

Switch#copu running-config startup-config
^
% Invalid input detected at '^' marker.

Switch#copy running-config startup-config
Destination filename [startup-config]?
Building configuration...
[OK]
Switch#

```

c) Now assign IP to the PCs and try pining the PC in different subnet



```
C:\>ping 192.168.2.1

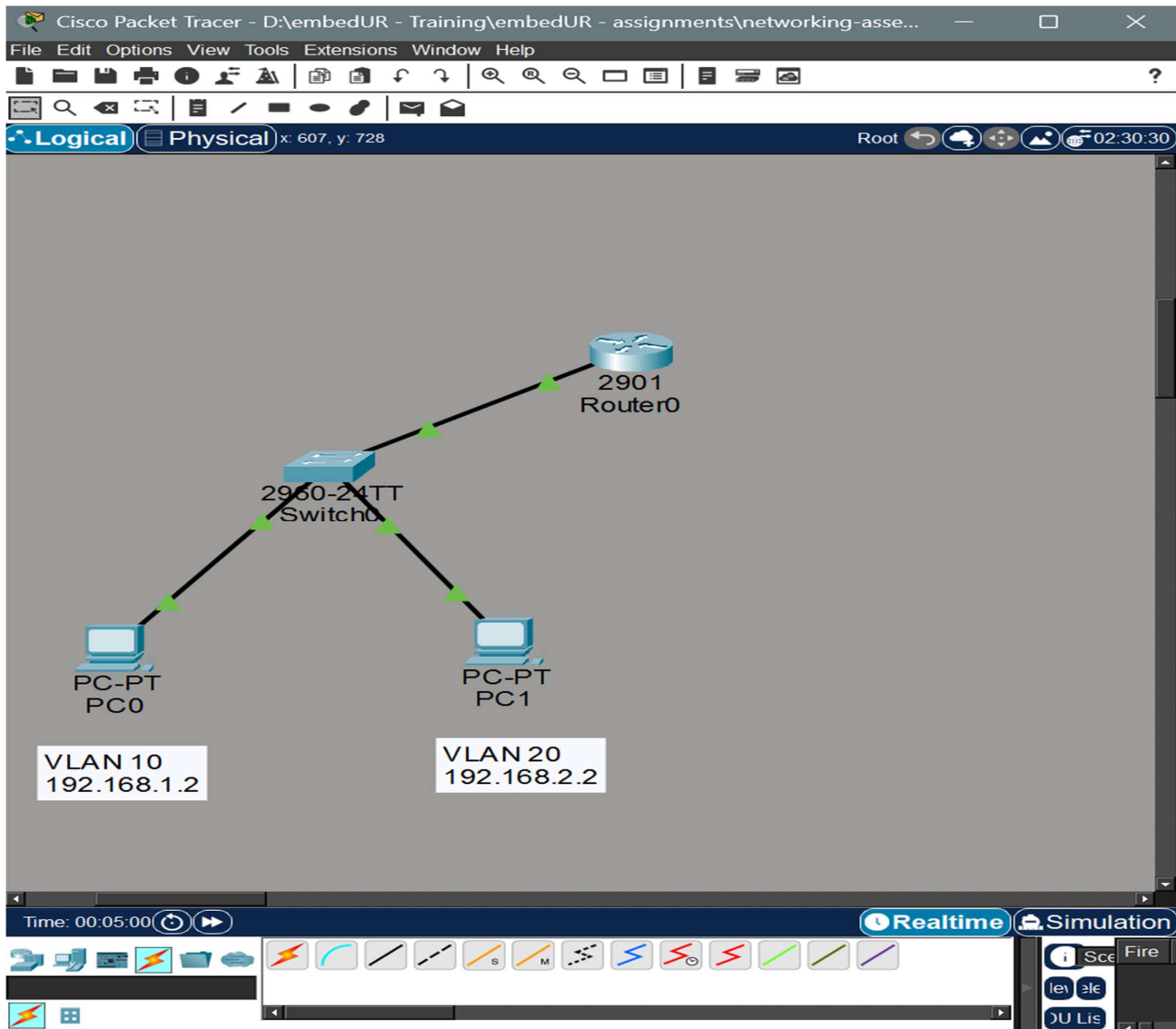
Pinging 192.168.2.1 with 32 bytes of data:

Reply from 192.168.2.1: bytes=32 time<1ms TTL=255
Reply from 192.168.2.1: bytes=32 time<1ms TTL=255
Reply from 192.168.2.1: bytes=32 time<1ms TTL=255
Reply from 192.168.2.1: bytes=32 time<1ms TTL=255

Ping statistics for 192.168.2.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 0ms, Average = 0ms
```

10. Try inter-VLAN routing using Router

I have used the following topology to try inter-VLAN routing using Router



a) Configure the switch, create the corresponding VLANs and assign them to the particular ports.

```
Switch>en
Switch#conf t
Enter configuration commands, one per line. End with CNTL/Z.
Switch(config)#vlan 10
Switch(config-vlan)#vlan 20
Switch(config-vlan)#do show vlan brief
```

VLAN	Name	Status	Ports
1	default	active	Fa0/1, Fa0/2, Fa0/3, Fa0/4 Fa0/5, Fa0/6, Fa0/7, Fa0/8 Fa0/9, Fa0/10, Fa0/11, Fa0/12 Fa0/13, Fa0/14, Fa0/15, Fa0/16 Fa0/17, Fa0/18, Fa0/19, Fa0/20 Fa0/21, Fa0/22, Fa0/23, Fa0/24 Gig0/1, Gig0/2
10	VLAN0010	active	
20	VLAN0020	active	
1002	fddi-default	active	
1003	token-ring-default	active	
1004	fddinet-default	active	
1005	trnet-default	active	

```
Switch(config-vlan)#
```

```
Switch(config)#int fa0/1
Switch(config-if)#switchport mode access
Switch(config-if)#switchport access vlan 10
Switch(config-if)#no shutdown
Switch(config-if)#exit
Switch(config)#int fa0/2
Switch(config-if)#switchport access vlan 20
Switch(config-if)#no shutdown
Switch(config-if)#exit
Switch(config)#do show vlan brief
```

VLAN	Name	Status	Ports
1	default	active	Fa0/3, Fa0/4, Fa0/5, Fa0/6 Fa0/7, Fa0/8, Fa0/9, Fa0/10 Fa0/11, Fa0/12, Fa0/13, Fa0/14 Fa0/15, Fa0/16, Fa0/17, Fa0/18 Fa0/19, Fa0/20, Fa0/21, Fa0/22 Fa0/23, Fa0/24, Gig0/1, Gig0/2
10	VLAN0010	active	Fa0/1
20	VLAN0020	active	Fa0/2
1002	fddi-default	active	
1003	token-ring-default	active	
1004	fddinet-default	active	
1005	trnet-default	active	

```
Switch(config)#
```

```
Switch(config)#int fa0/3
Switch(config-if)#switchport mode trunk
```

```
Switch(config-if)#
%LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet0/3, changed state to down

%LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet0/3, changed state to up
```

b) Now configure the router and create sub interfaces

```

Router>ena
Router#conf
Router#conf terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#int g
Router(config)#int gigabitEthernet 0/0
Router(config-if)#no shutdown
Router(config-if)#interface g0/0.10
Router(config-subif)#
%LINK-3-UPDOWN: Interface GigabitEthernet0/0.10, changed state to down

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0.10, changed state to up

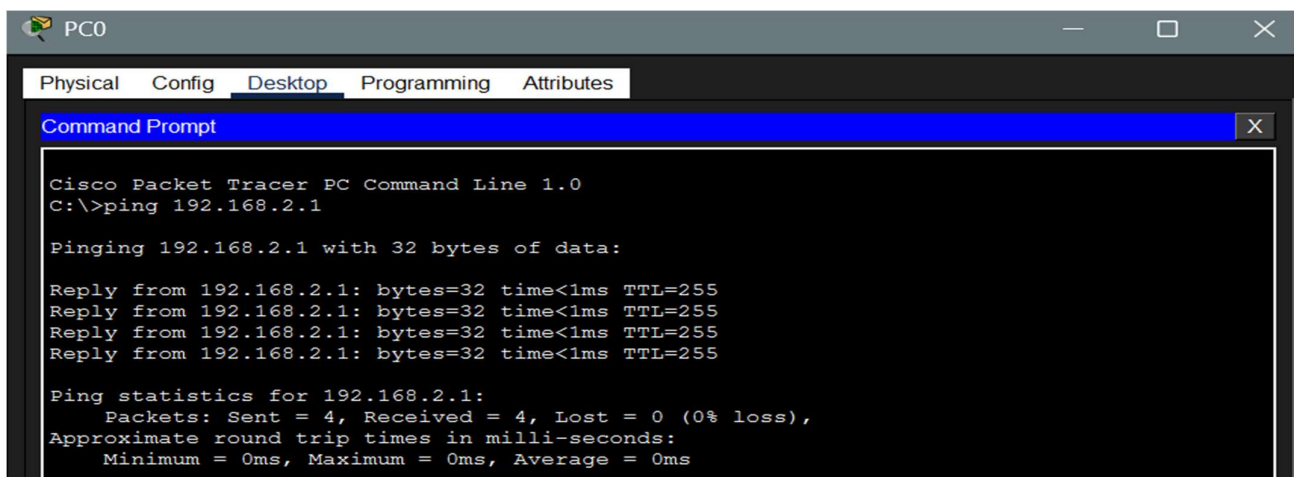
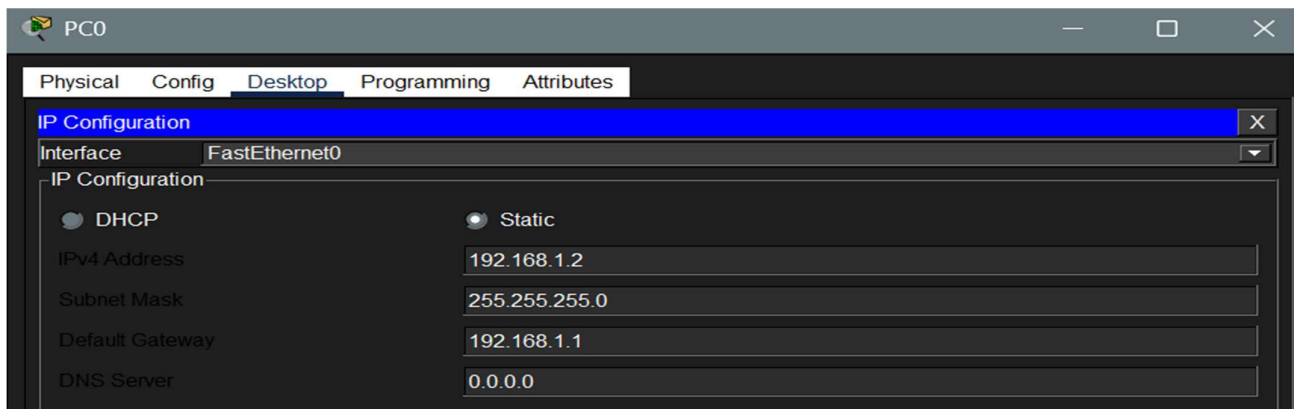
Router(config-subif)#encapsulation dot1Q 10
Router(config-subif)#ip address 192.168.1.1 255.255.255.0
Router(config-subif)#interface g0/0.20
Router(config-subif)#
%LINK-3-UPDOWN: Interface GigabitEthernet0/0.20, changed state to down

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0.20, changed state to up

Router(config-subif)#encapsulation dot1Q 20
Router(config-subif)#ip address 192.168.2.1 255.255.255.0
Router(config-subif)#

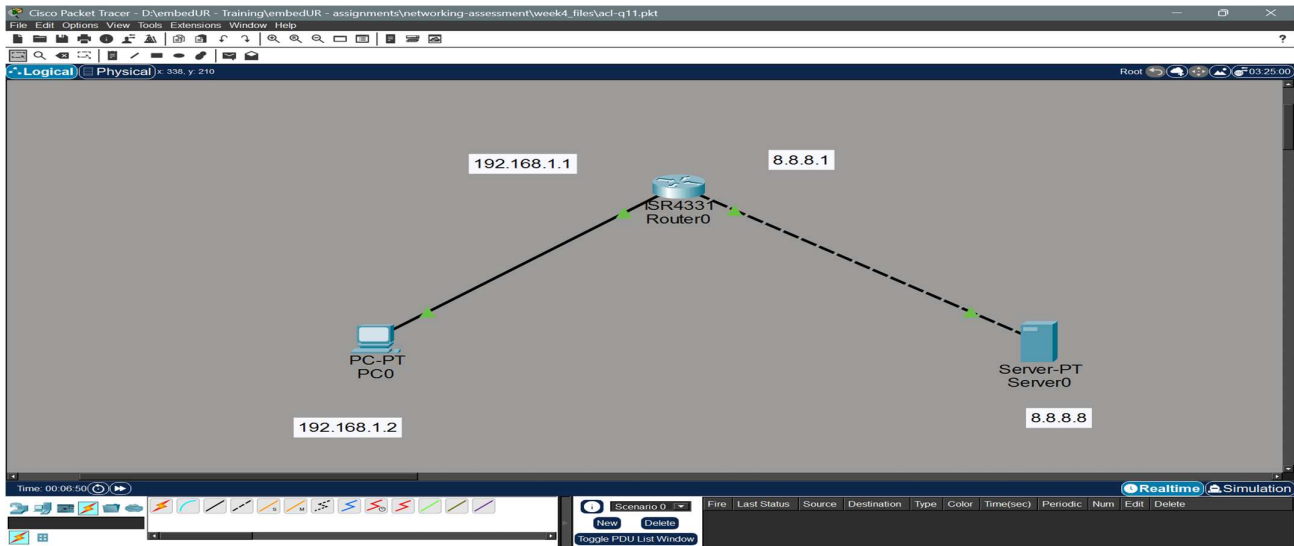
```

c) Now configure IP and Gateway in each PCs

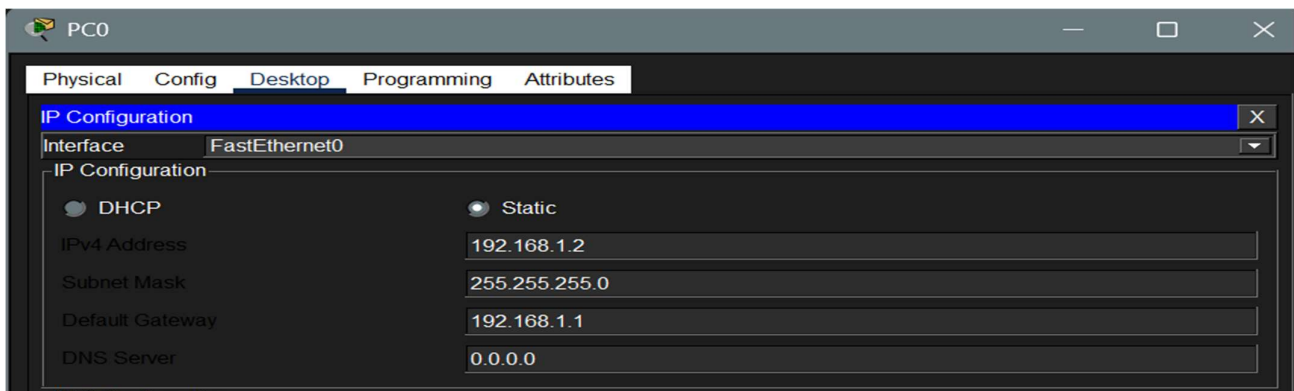


11. Implement ACL to restrict traffic based in source and destination ports. Test rules by simulating legitimate and unauthorized traffic.

To implement ACL in a router we use the following topology



a) Assign IP address and Gateway address to the PC, Server and assign IP to the router Interfaces.

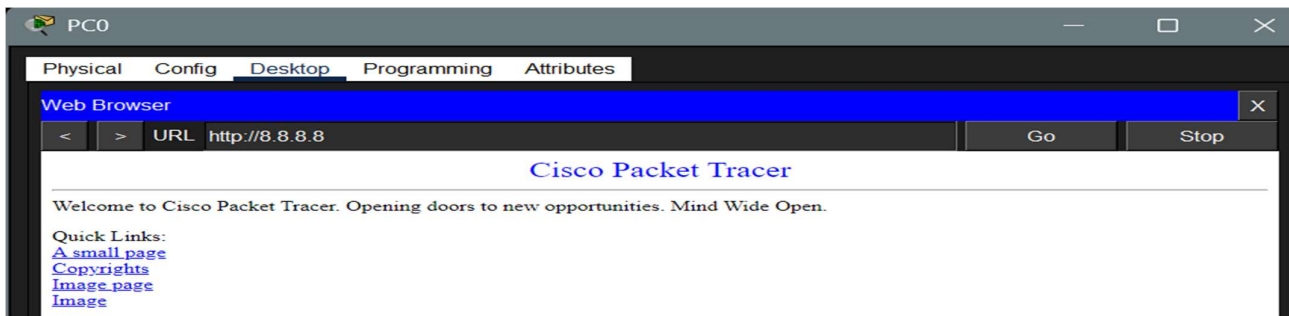


```
Router(config)#int gig0/0/0
Router(config-if)#ip address 192.168.1.1 255.255.255.0
Router(config-if)#exit
Router(config)#int gig0/0/1
Router(config-if)#ip address 8.8.8.1 255.0.0.0
Router(config-if)#exit
Router(config)#end
```

b) Configure the ACL using access-list command in the router. I am going to

- i) Block traffic to port 80 (HTTP)
- ii) Allow traffic to port 443 (HTTPS)

(Before doing this turn on HTTP & HTTPS services in the router)



We need to create an extended ACL,

```
Router>enable
Router#conf t
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#access
Router(config)#access-list ?
    <1-99>      IP standard access list
    <100-199>   IP extended access list
Router(config)#access-list

Router(config)#access-list 101 deny tcp 192.168.1.0 0.0.0.255 host 8.8.8.8 eq 80
Router(config)#access-list 101 allow tcp 192.168.1.0 0.0.0.255 host 8.8.8.8 eq 443
      ^
% Invalid input detected at '^' marker.

Router(config)#access-list 101 permit tcp 192.168.1.0 0.0.0.255 host 8.8.8.8 eq 443
Router(config)#access-list 101 permit ip any any

Router#show acce
Router#show access-lists
Extended IP access list 101
    10 deny tcp 192.168.1.0 0.0.0.255 host 8.8.8.8 eq www
    20 permit tcp 192.168.1.0 0.0.0.255 host 8.8.8.8 eq 443
    30 permit ip any any

Router#

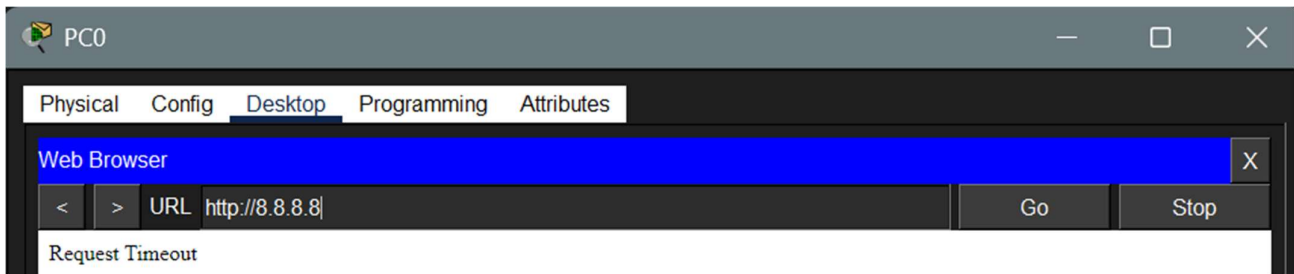
Router(config)#int gig0/0/1
Router(config-if)#ip access-list 101
      ^
% Invalid input detected at '^' marker.

Router(config-if)#ip acces
Router(config-if)#exit
Router(config)#int gig0/0/1
Router(config-if)#ip access-group 101
% Incomplete command.

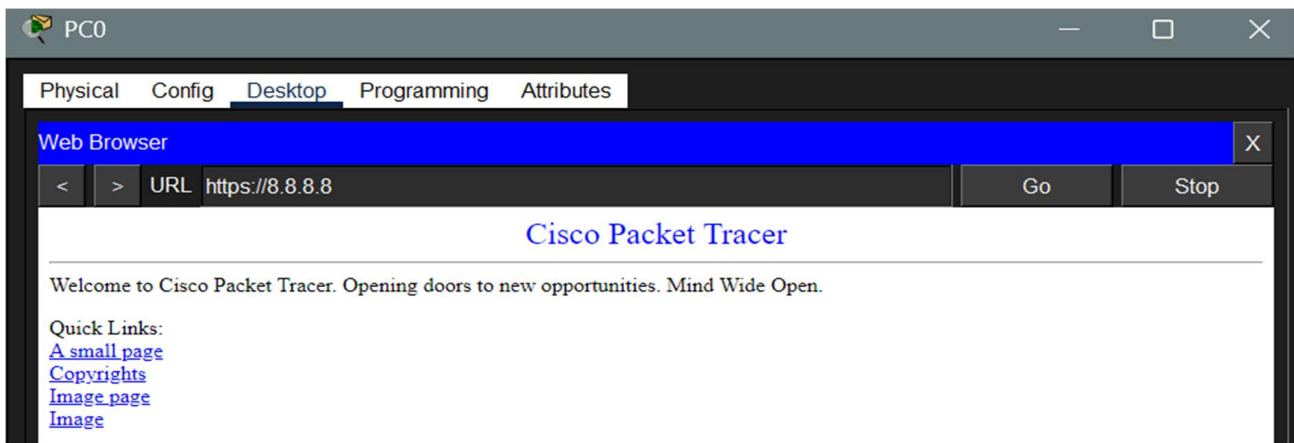
Router(config-if)#ip access-group 101 out
```


c) Now check the web server access from the client device.

Http access:

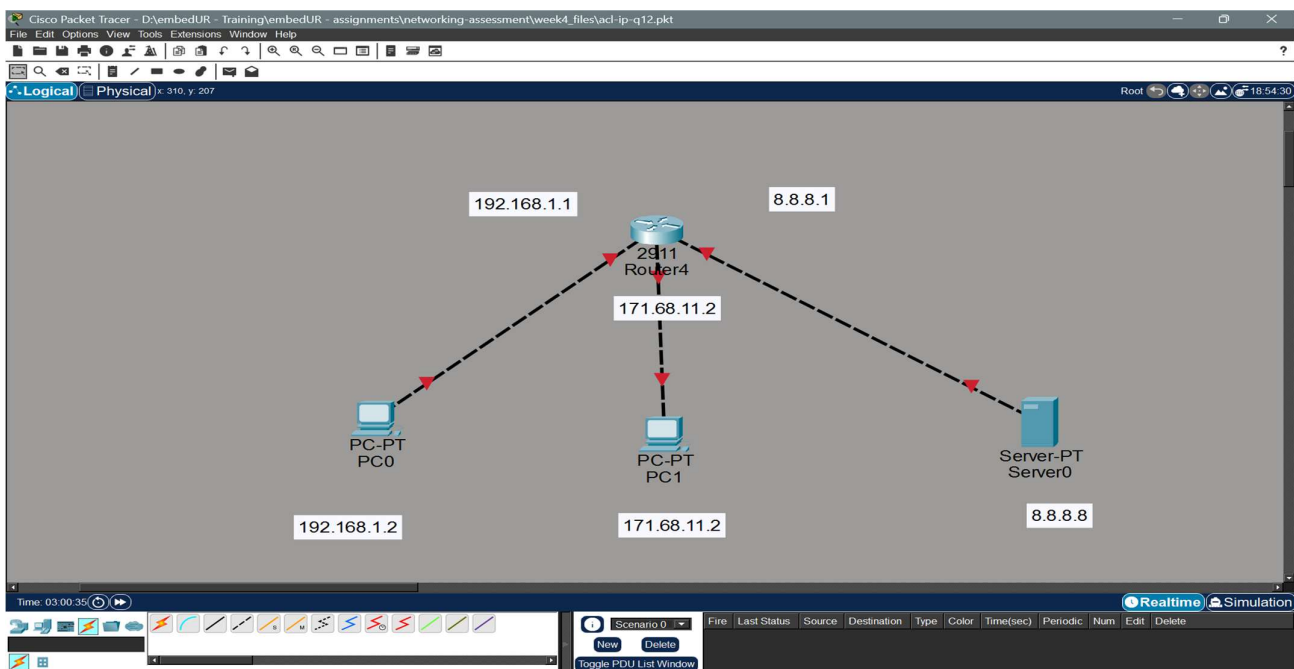


Https access:

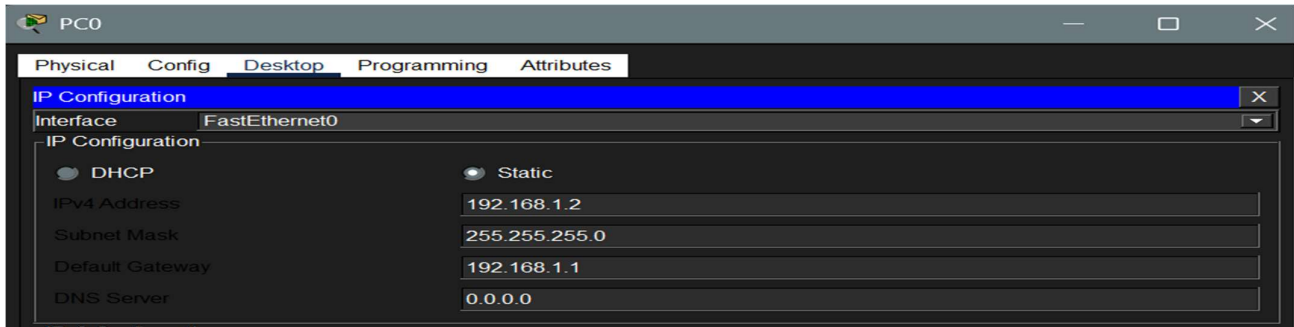


12) Configure a standard ACL on a router to allow traffic from a particular specific range of IP. Test connectivity and verify that the ACL is working as intended.

I have used the following topology fir this experiment,



a) Assign IP to the PCs and the router interfaces



```
Router>en
Router#conf t
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#int gig0/0
Router(config-if)#no shutdown

Router(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/0, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to up

Router(config-if)#ip address 192.168.1.1 255.255.255.0
Router(config-if)#exit
Router(config)#int gig0/1
Router(config-if)#no shutdown

Router(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/1, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/1, changed state to up

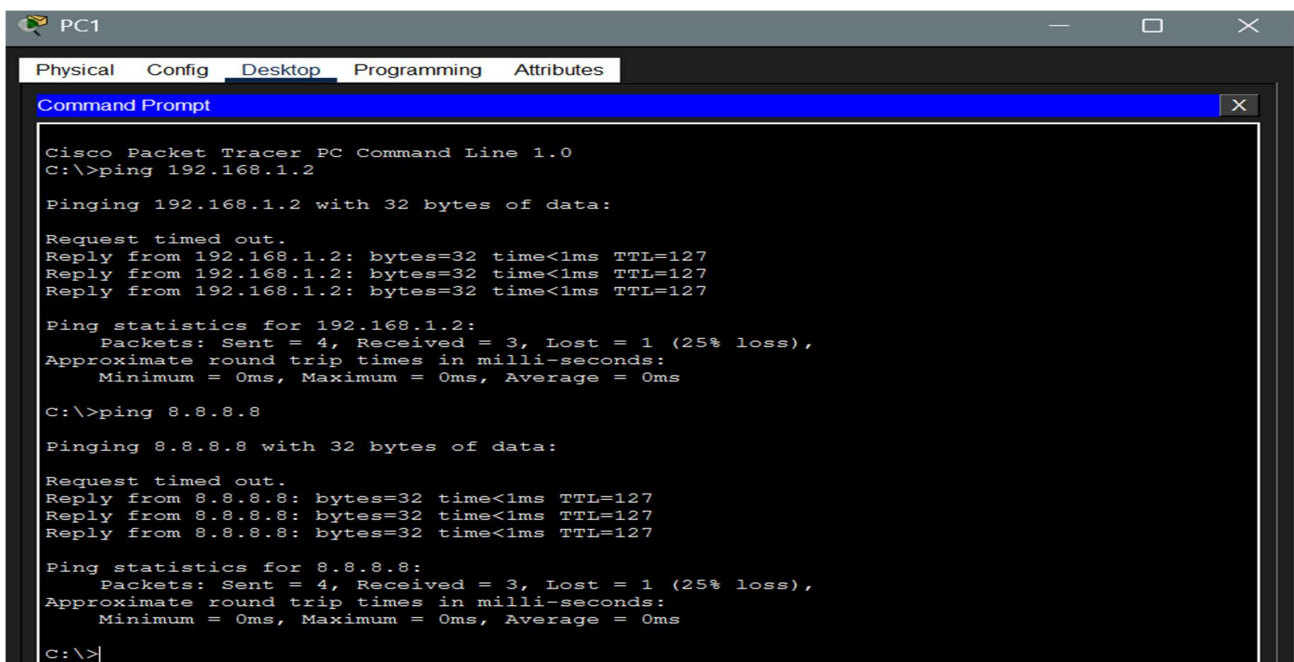
Router(config-if)#ip address 172.68.11.1 255.255.0.0
Router(config-if)#exit
Router(config)#int gig0/3
%Invalid interface type and number
Router(config)#int gig0/2
Router(config-if)#no shutdown

Router(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/2, changed state to up

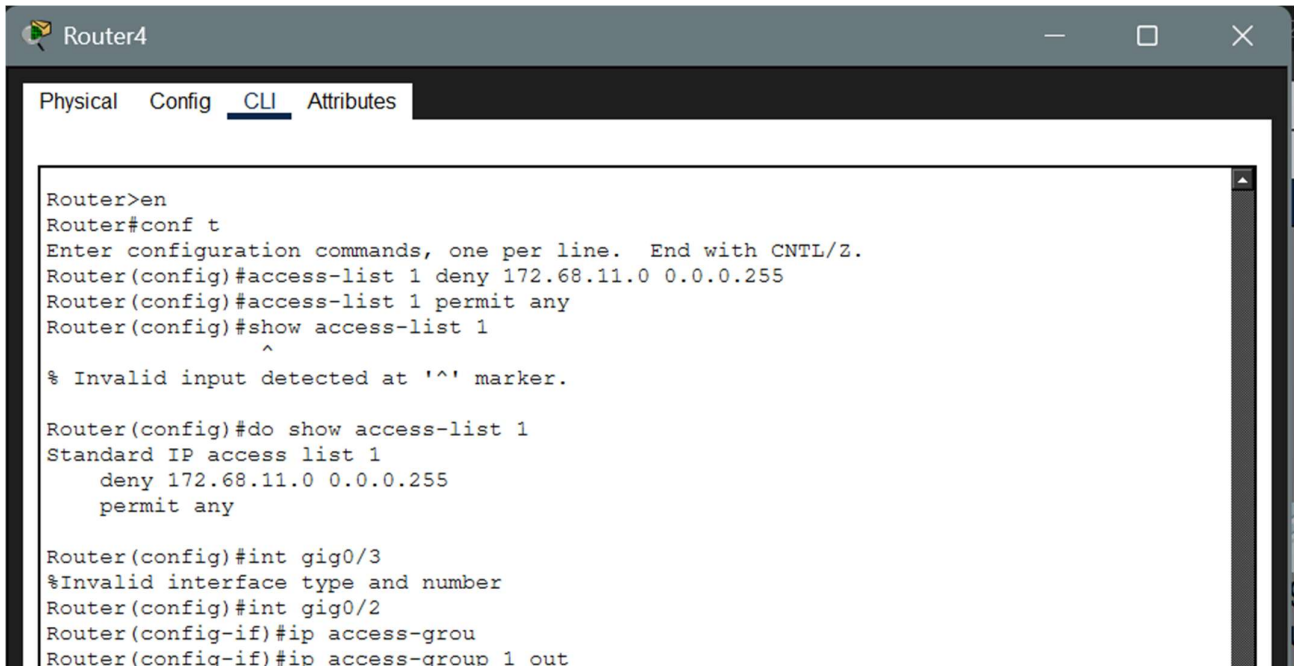
%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/2, changed state to up

Router(config-if)#ip address 8.8.8.1 255.0.0.0
Router(config-if)#exit
Router(config)#
```

b) Try whether they are reachable.



c) Now configure ACL using access-list command to block the access from a particular range of IPs assume I am blocking access from 172.68.11.X to the server.



The screenshot shows the CLI of Router4. The user enters 'en' to enter enable mode, then 'conf t' to enter configuration mode. They create an access-list 1 that denies traffic from 172.68.11.0/24 and permits any other traffic. They then attempt to apply this ACL to gig0/3, which fails due to an invalid interface number. Finally, they apply the ACL to gig0/2.

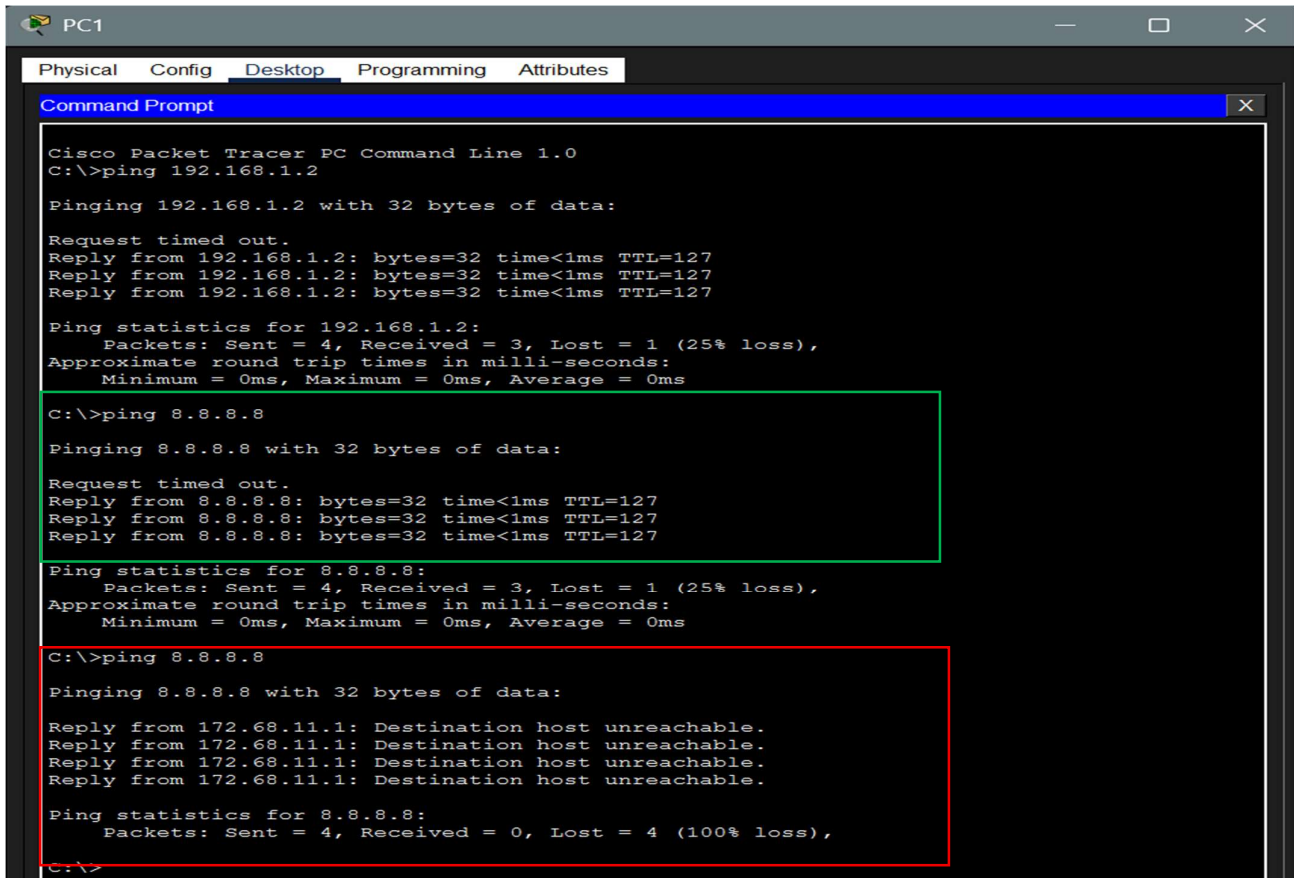
```

Router>en
Router#conf t
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#access-list 1 deny 172.68.11.0 0.0.0.255
Router(config)#access-list 1 permit any
Router(config)#show access-list 1
      ^
% Invalid input detected at '^' marker.

Router(config)#do show access-list 1
Standard IP access list 1
    deny 172.68.11.0 0.0.0.255
    permit any

Router(config)#int gig0/3
%Invalid interface type and number
Router(config)#int gig0/2
Router(config-if)#ip access-group
Router(config-if)#ip access-group 1 out
  
```

d) Now check the connectivity to the server from the PC1 to the Subnet 8.8.8.0/8



The screenshot shows the Command Prompt on PC1. It displays the results of three ping commands. The first two pings are to 192.168.1.2 and 8.8.8.8, both showing successful connectivity with 3 successful replies and 1 loss (25% loss). The third ping is to 8.8.8.8, which shows no connectivity, with all 4 packets lost (100% loss).

```

Cisco Packet Tracer PC Command Line 1.0
C:\>ping 192.168.1.2

Pinging 192.168.1.2 with 32 bytes of data:

Request timed out.
Reply from 192.168.1.2: bytes=32 time<1ms TTL=127
Reply from 192.168.1.2: bytes=32 time<1ms TTL=127
Reply from 192.168.1.2: bytes=32 time<1ms TTL=127

Ping statistics for 192.168.1.2:
    Packets: Sent = 4, Received = 3, Lost = 1 (25% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 0ms, Average = 0ms

C:\>ping 8.8.8.8

Pinging 8.8.8.8 with 32 bytes of data:

Request timed out.
Reply from 8.8.8.8: bytes=32 time<1ms TTL=127
Reply from 8.8.8.8: bytes=32 time<1ms TTL=127
Reply from 8.8.8.8: bytes=32 time<1ms TTL=127

Ping statistics for 8.8.8.8:
    Packets: Sent = 4, Received = 3, Lost = 1 (25% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 0ms, Average = 0ms

C:\>ping 8.8.8.8

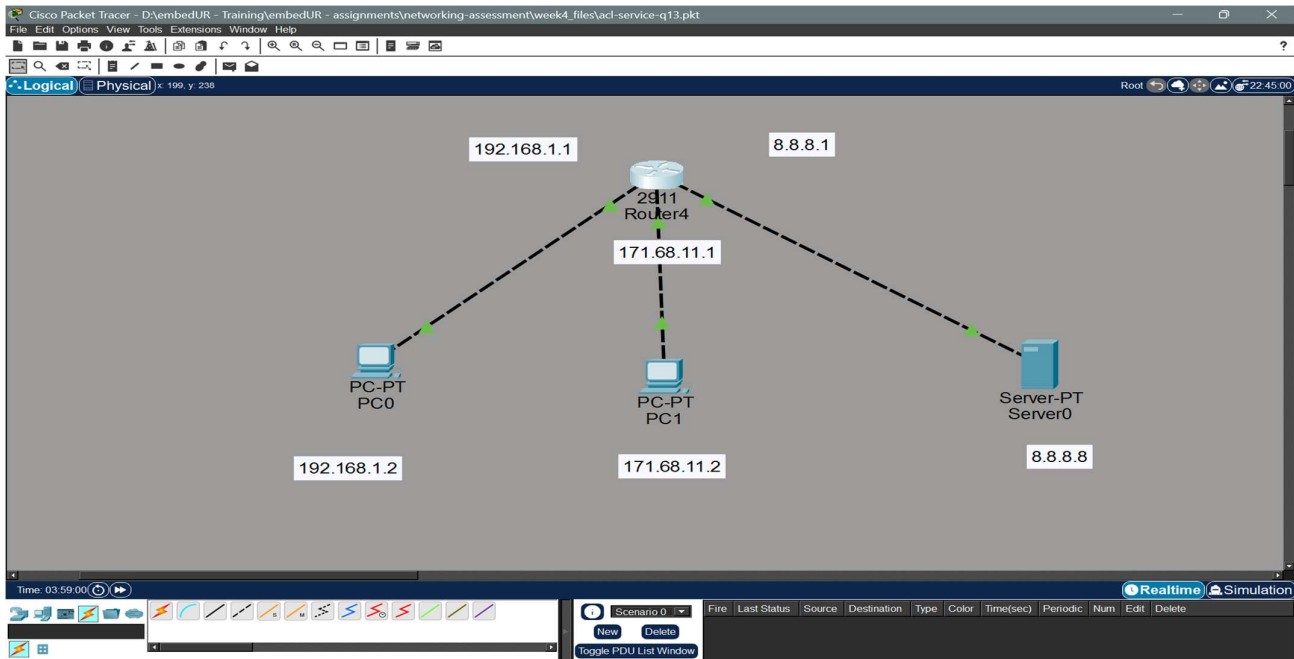
Pinging 8.8.8.8 with 32 bytes of data:

Reply from 172.68.11.1: Destination host unreachable.
Reply from 172.68.11.1: Destination host unreachable.
Reply from 172.68.11.1: Destination host unreachable.
Reply from 172.68.11.1: Destination host unreachable.

Ping statistics for 8.8.8.8:
    Packets: Sent = 4, Received = 0, Lost = 4 (100% loss),
  
```

13) Create an extended ACL to block specific applications, such as HTTP or FTP traffic. Test the ACL rule by attempting to access the blocked services.

I have used the same topology and configuration as above.

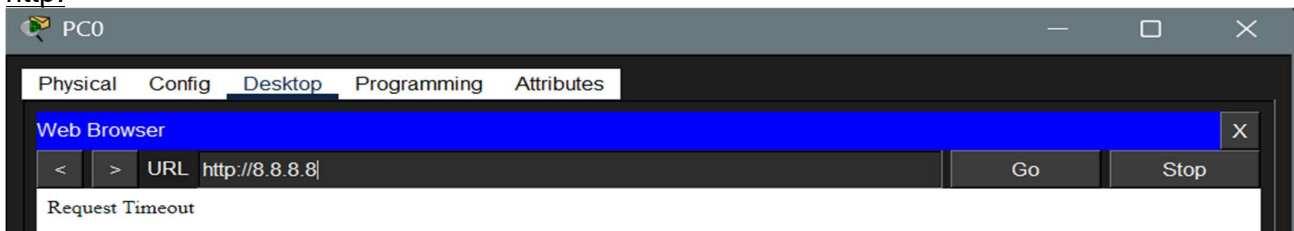


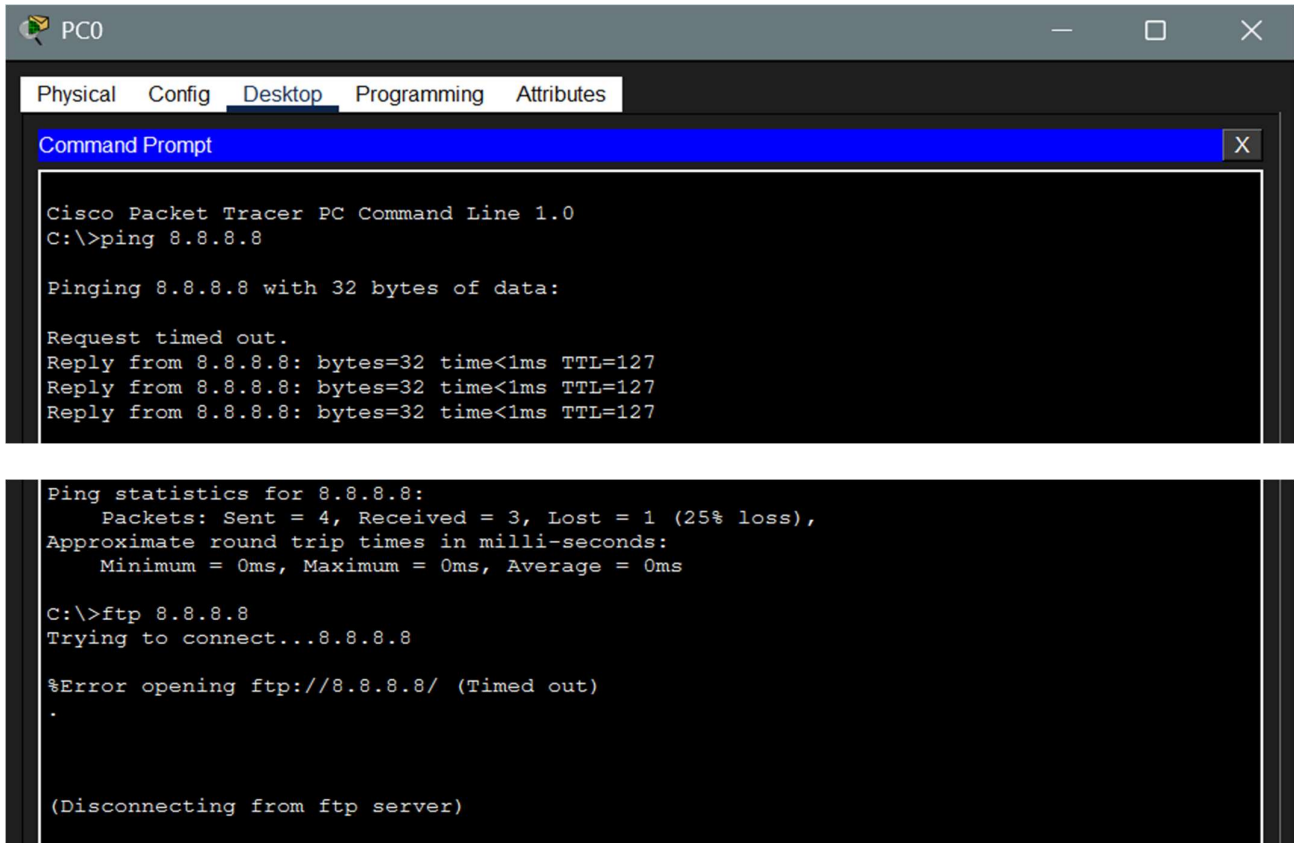
a) Start the HTTP / FTP service in the server and create the access-list to block it

```
Router(config)#access-list 101 deny tcp any any eq 80
Router(config)#access-list 101 deny tcp any any eq 21
Router(config)#access-list 101 deny tcp any any eq 20
Router(config)#access-list 101 permit ip any any
Router(config)#int gig0/2
Router(config-if)#ip access-group 101 out
Router(config-if)#do show access-list
Extended IP access list 101
 10 deny tcp any any eq www
 20 deny tcp any any eq ftp
 30 deny tcp any any eq 20
 40 permit ip any any
```

b) Check the connection

http:



FTP (can access but FTP fails)


```

PC0
Physical Config Desktop Programming Attributes
Command Prompt
Cisco Packet Tracer PC Command Line 1.0
C:\>ping 8.8.8.8

Pinging 8.8.8.8 with 32 bytes of data:

Request timed out.
Reply from 8.8.8.8: bytes=32 time<1ms TTL=127
Reply from 8.8.8.8: bytes=32 time<1ms TTL=127
Reply from 8.8.8.8: bytes=32 time<1ms TTL=127

Ping statistics for 8.8.8.8:
    Packets: Sent = 4, Received = 3, Lost = 1 (25% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 0ms, Average = 0ms

C:\>ftp 8.8.8.8
Trying to connect...8.8.8.8

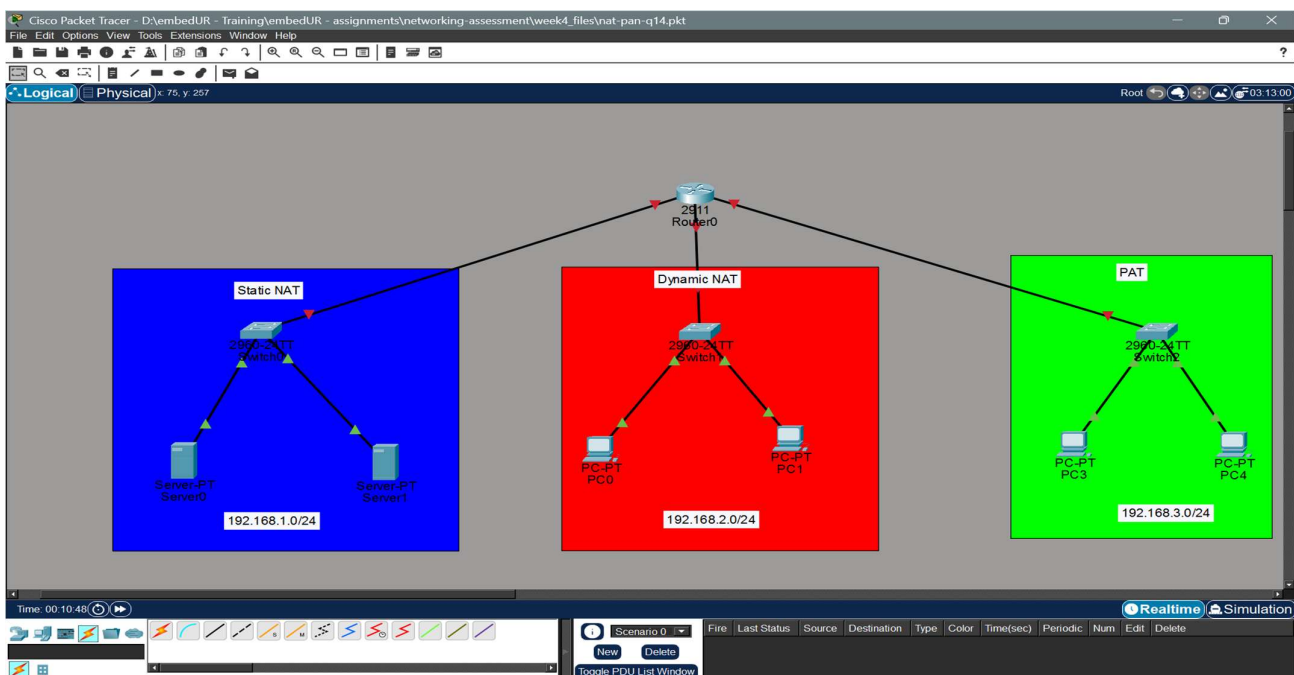
%Error opening ftp://8.8.8.8/ (Timed out)
.

(Disconnecting from ftp server)

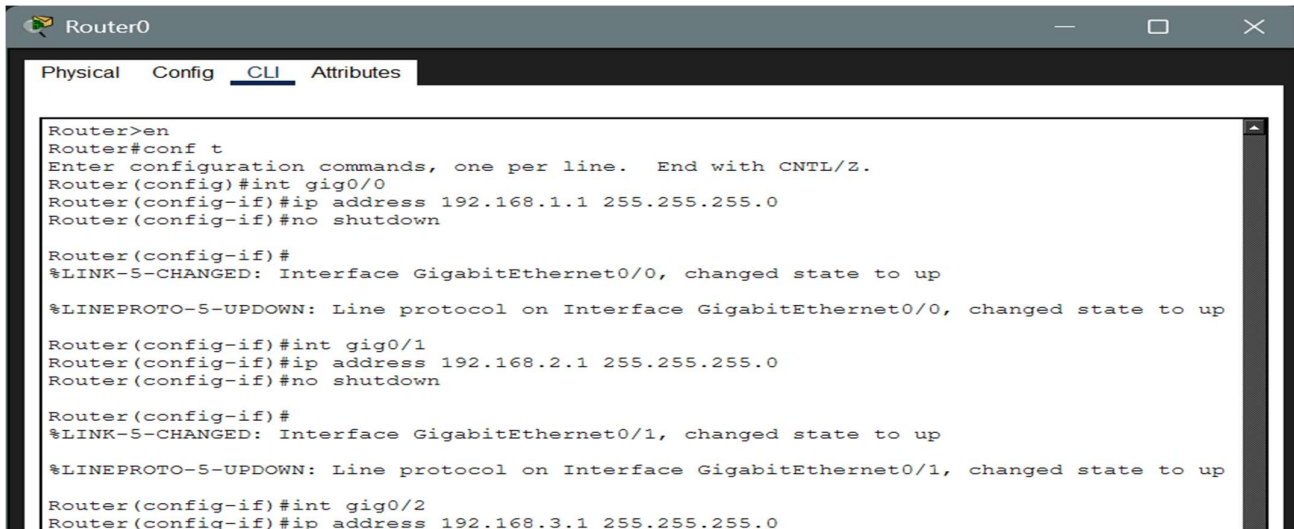
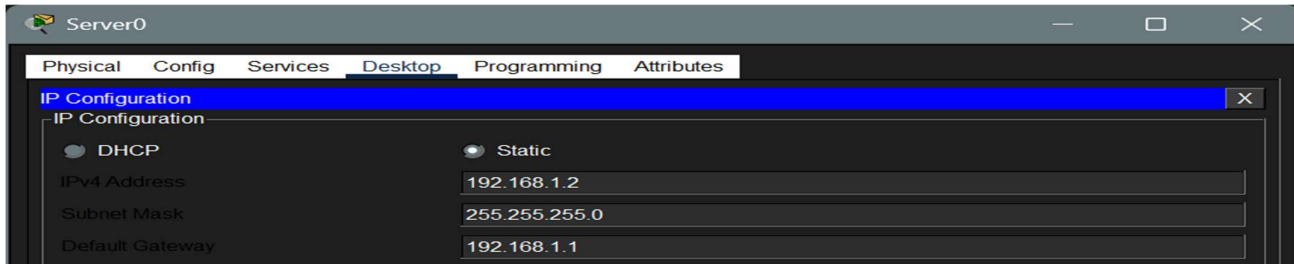
```

14) Try static NAT, Dynamic NAT and PAT to translate IPs

I have used the following topology for this experiment:



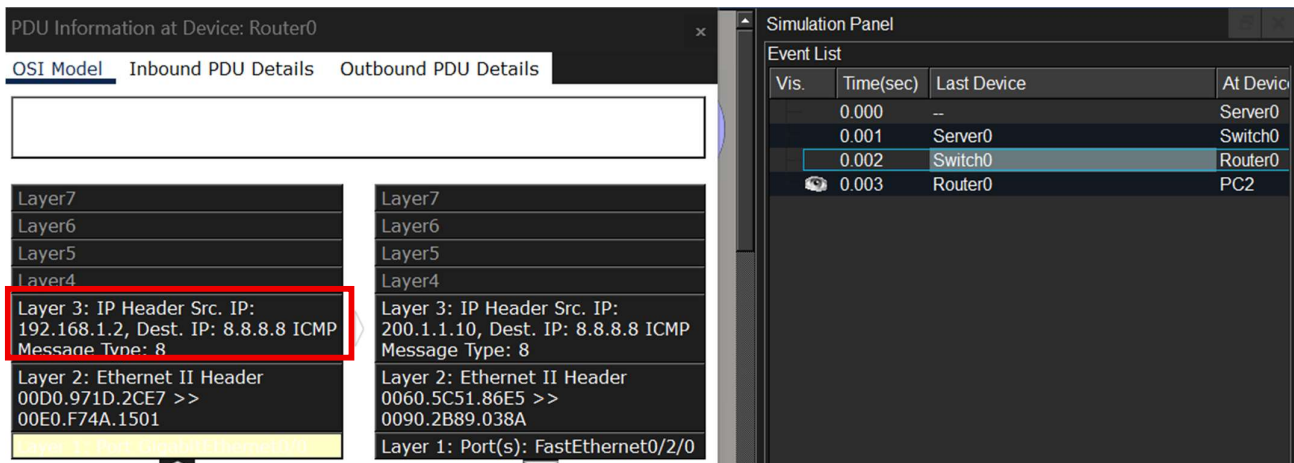
Assign IP to each interface, the PCs and the servers



a) Static NAT:

```
Router#en
Router#conf t
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#int g0/0
Router(config-if)#ip nat inside
Router(config-if)#ip nat inside source static 192.168.1.2 200.1.1.10
Router(config)#ip nat inside source static 192.168.1.3 200.1.1.11
Router(config)#
```

Test from WAN: (Before Reaching router & After reaching Router)



The screenshot displays the 'PDU Information at Device: PC2' window with the 'Inbound PDU Details' tab selected. The PDU details are as follows:

Layer	Details
Layer 7	
Layer 6	
Layer 5	
Layer 4	
Layer 3	IP Header Src. IP: 200.1.1.10, Dest. IP: 8.8.8.8 ICMP Message Type: 8
Layer 2	Ethernet II Header 0060.5C51.86E5 >> 0090.2B89.038A
Layer 1	Port(s): FastEthernet0

The 'Simulation Panel' on the right shows the 'Event List' with the following entries:

Vis.	Time(sec)	Last Device	At Device
	0.000	-	Server0
	0.001	Server0	Switch0
	0.002	Switch0	Router0
	0.003	Router0	PC2

b) Dynamic NAT:

```
Router>en
Router#conf t
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#access-list 1 permit 192.168.2.0 0.0.0.255
Router(config)#ip nat pool NAT_POOL 8.8.8.20 8.8.8.30 netmask 255.0.0.0
Router(config)#ip nat inside source list 1 pool NAT_POOL
```

Testing from WAN (using NAT table)

The screenshot shows the 'NAT Table for Router0' window with the following data:

Protocol	Inside Global	Inside Local	Outside Local	Outside Global
icmp	200.1.1.10.3	192.168.1.2.3	8.8.8.8.3	8.8.8.8.3
---	200.1.1.10	192.168.1.2	---	---
---	200.1.1.11	192.168.1.3	---	---

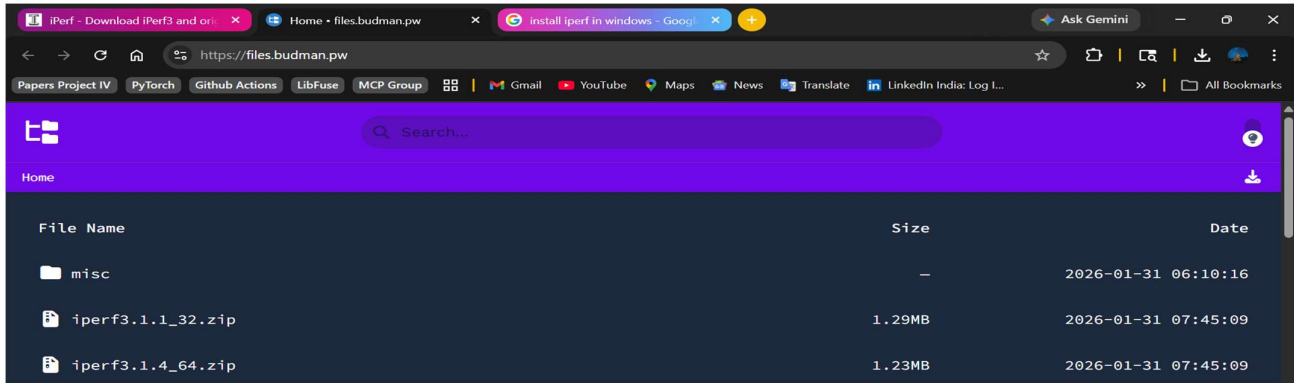
The background shows a network diagram with three routers (Static NAT, Dynamic NAT, and PAT) and their respective connected devices (PCs and Switches).

PAT:

```
Router#conf t
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#int g0/2
Router(config-if)#ip nat inside
Router(config-if)#access-list 2 permit 192.168.3.0 0.0.0.255
Router(config)#ip nat inside source list 2 interface fa0/2/0 overload
```

15) Download iperf in laptop/phone and make sure they are in same network. Try different iperf commands with tcp, udp, bidirectional, reverse, multicast, parrel options and analyze the bandwidth and rate of transmission, delay, jitter etc.,

a) Install iperf from the official site and open the cmd prompt in the same folder



b) In the server and clint system enter (ensure they are in same subnet)

i. iperf -s (in server system)

ii. iperf -c <ip of server> (in client system)

```

C:\Windows\System32\cmd.e  x  +  v

=====
Hello Mystery Chap
=====

"Compiling life.exe... Error: Too many exceptions."

=====
Today: 22-03-2026      Time: 19:20:09.26
User: Mystery Chap
=====

D:\embedUR - Training\embedUR Networking\iperf3.1.4_64>iperf3.exe -c 10.161.138.202
Connecting to host 10.161.138.202, port 5201
[ 4] local 10.161.138.50 port 56742 connected to 10.161.138.202 port 5201
[ ID] Interval           Transfer     Bandwidth
[ 4]  0.00-1.00      sec    2.00 MBytes   16.7 Mbits/sec
[ 4]  1.00-2.01      sec    1.88 MBytes   15.6 Mbits/sec
[ 4]  2.01-3.01      sec    1.75 MBytes   14.6 Mbits/sec
[ 4]  3.01-4.01      sec    2.00 MBytes   16.9 Mbits/sec
[ 4]  4.01-5.00      sec    2.00 MBytes   16.9 Mbits/sec
[ 4]  5.00-6.01      sec    1.88 MBytes   15.6 Mbits/sec
[ 4]  6.01-7.00      sec    1.75 MBytes   14.8 Mbits/sec
[ 4]  7.00-8.01      sec    1.75 MBytes   14.6 Mbits/sec
[ 4]  8.01-9.01      sec    1.75 MBytes   14.7 Mbits/sec
[ 4]  9.01-10.01     sec    2.00 MBytes   16.7 Mbits/sec
- - - - -
[ ID] Interval           Transfer     Bandwidth
[ 4]  0.00-10.01     sec   18.8 MBytes   15.7 Mbits/sec  sender
[ 4]  0.00-10.01     sec   18.6 MBytes   15.6 Mbits/sec receiver

iperf Done.

D:\embedUR - Training\embedUR Networking\iperf3.1.4_64>

```

The above image is the TCP based results

c) For UDP we use, the command with following flags, `iperf -c <ip> -u -b <size>`

```
D:\embedUR - Training\embedUR Networking\iperf3.1.4_64>iperf3.exe -c 10.161.138.202 -u -b 1M
Connecting to host 10.161.138.202, port 5201
[ 4] local 10.161.138.50 port 64188 connected to 10.161.138.202 port 5201
[ ID] Interval      Transfer      Bandwidth      Total Datagrams
[ 4] 0.00-1.00    sec    112 KBytes    917 Kbits/sec    14
[ 4] 1.00-2.01    sec    128 KBytes    1.04 Mbits/sec   16
[ 4] 2.01-3.00    sec    120 KBytes    987 Kbits/sec   15
[ 4] 3.00-4.00    sec    120 KBytes    983 Kbits/sec   15
[ 4] 4.00-5.00    sec    120 KBytes    982 Kbits/sec   15
[ 4] 5.00-6.00    sec    128 KBytes    1.05 Mbits/sec   16
[ 4] 6.00-7.00    sec    120 KBytes    982 Kbits/sec   15
[ 4] 7.00-8.00    sec    120 KBytes    983 Kbits/sec   15
[ 4] 8.00-9.01    sec    120 KBytes    972 Kbits/sec   15
[ 4] 9.01-10.01   sec    128 KBytes    1.05 Mbits/sec   16
- - - - -
[ ID] Interval      Transfer      Bandwidth      Jitter    Lost/Total Datagrams
[ 4] 0.00-10.01   sec    1.19 MBytes    995 Kbits/sec   41.457 ms  0/152 (0%)
[ 4] Sent 152 datagrams

iperf Done.
```

d) For reverse test (client connection test) we use the command with following flags, `iperf -c <ip> -R`

```
D:\embedUR - Training\embedUR Networking\iperf3.1.4_64>iperf3.exe -c 10.161.138.202 -R
Connecting to host 10.161.138.202, port 5201
Reverse mode, remote host 10.161.138.202 is sending
[ 4] local 10.161.138.50 port 60039 connected to 10.161.138.202 port 5201
[ ID] Interval      Transfer      Bandwidth
[ 4] 0.00-1.00    sec    2.32 MBytes    19.4 Mbits/sec
[ 4] 1.00-2.00    sec    2.25 MBytes    19.0 Mbits/sec
[ 4] 2.00-3.00    sec    2.01 MBytes    16.9 Mbits/sec
[ 4] 3.00-4.00    sec    2.09 MBytes    17.5 Mbits/sec
[ 4] 4.00-5.01    sec    1.86 MBytes    15.5 Mbits/sec
[ 4] 5.01-6.00    sec    1.96 MBytes    16.6 Mbits/sec
[ 4] 6.00-7.01    sec    2.27 MBytes    18.9 Mbits/sec
[ 4] 7.01-8.00    sec    2.13 MBytes    18.0 Mbits/sec
[ 4] 8.00-9.00    sec    2.34 MBytes    19.6 Mbits/sec
[ 4] 9.00-10.00   sec    2.36 MBytes    19.8 Mbits/sec
- - - - -
[ ID] Interval      Transfer      Bandwidth
[ 4] 0.00-10.00   sec    21.7 MBytes    18.2 Mbits/sec    sender
[ 4] 0.00-10.00   sec    21.7 MBytes    18.2 Mbits/sec    receiver

iperf Done.
```

e) For parallel test we use `iperf -c <ip> -P <no of parallel streams>`

```
D:\embedUR - Training\embedUR Networking\iperf3.1.4_64>iperf3.exe -c 10.161.138.202 -P 5
Connecting to host 10.161.138.202, port 5201
[ 4] local 10.161.138.50 port 51678 connected to 10.161.138.202 port 5201
[ 6] local 10.161.138.50 port 51679 connected to 10.161.138.202 port 5201
[ 8] local 10.161.138.50 port 51680 connected to 10.161.138.202 port 5201
[10] local 10.161.138.50 port 51681 connected to 10.161.138.202 port 5201
[12] local 10.161.138.50 port 51682 connected to 10.161.138.202 port 5201
[ ID] Interval      Transfer      Bandwidth
[ 4] 0.00-1.01    sec    768 KBytes    6.22 Mbits/sec
[ 6] 0.00-1.01    sec    512 KBytes    4.15 Mbits/sec
[ 8] 0.00-1.01    sec    512 KBytes    4.15 Mbits/sec
[10] 0.00-1.01    sec    640 KBytes    5.19 Mbits/sec
[12] 0.00-1.01    sec    640 KBytes    5.19 Mbits/sec
[SUM] 0.00-1.01   sec    3.00 MBytes    24.9 Mbits/sec
```

f) To do multicast analysis we need iperf 2 as iperf 3 doesn't have explicit support for multicast testing

```
D:\embedUR - Training\embedUR Networking\iperf3.1.4_64>iperf-2.exe -c 10.161.138.202 -u -T 3 -t 20 -i 1 -b 100M
-----
Client connecting to 10.161.138.202, UDP port 5001
Sending 1470 byte datagrams, IPG target: 0.00 us (kalman adjust)
UDP buffer size: 64.0 KByte (default)
-----
[ 1] local 10.161.138.50 port 59314 connected with 10.161.138.202 port 5001
[ ID] Interval      Transfer    Bandwidth
[ 1] 0.00-1.00 sec  2.57 MBytes 21.6 Mbits/sec
[ 1] 1.00-2.00 sec  2.29 MBytes 19.2 Mbits/sec
[ 1] 2.00-3.00 sec  2.57 MBytes 21.6 Mbits/sec
[ 1] 3.00-4.00 sec  2.47 MBytes 20.7 Mbits/sec
[ 1] 4.00-5.00 sec  2.68 MBytes 22.5 Mbits/sec
[ 1] 5.00-6.00 sec  2.54 MBytes 21.3 Mbits/sec
[ 1] 6.00-7.00 sec  2.53 MBytes 21.2 Mbits/sec
[ 1] 7.00-8.00 sec  2.64 MBytes 22.2 Mbits/sec
[ 1] 8.00-9.00 sec  2.52 MBytes 21.2 Mbits/sec
[ 1] 9.00-10.00 sec 2.63 MBytes 22.1 Mbits/sec
[ 1] 10.00-11.00 sec 2.50 MBytes 20.9 Mbits/sec
[ 1] 11.00-12.00 sec 2.56 MBytes 21.5 Mbits/sec
[ 1] 12.00-13.00 sec 2.70 MBytes 22.6 Mbits/sec
[ 1] 13.00-14.00 sec 2.58 MBytes 21.7 Mbits/sec
[ 1] 14.00-15.00 sec 2.57 MBytes 21.5 Mbits/sec
[ 1] 15.00-16.00 sec 2.39 MBytes 20.0 Mbits/sec
[ 1] 16.00-17.00 sec 2.41 MBytes 20.3 Mbits/sec
[ 1] 17.00-18.00 sec 2.61 MBytes 21.9 Mbits/sec
[ 1] 18.00-19.00 sec 2.45 MBytes 20.5 Mbits/sec
[ 1] 19.00-20.00 sec 2.65 MBytes 22.2 Mbits/sec
[ 1] 0.00-20.01 sec 50.9 MBytes 21.3 Mbits/sec
[ 1] Sent 36284 datagrams
[ 1] Server Report:
[ ID] Interval      Transfer    Bandwidth      Jitter    Lost/Total Datagrams
[ 1] 0.00-20.13 sec 50.9 MBytes 21.2 Mbits/sec  0.000 ms  0/36283 (0%)
D:\embedUR - Training\embedUR Networking\iperf3.1.4_64>
```

Inference on Bandwidth, Jitter and Loss

Bandwidth:

The observed bandwidth remained approximately around 20 MB/s under most test conditions. In TCP-based connections, the bandwidth was comparatively lower due to protocol mechanisms such as acknowledgments, flow control, and congestion control, which introduce additional overhead.

In parallel stream testing, the bandwidth was distributed across multiple streams, so the bandwidth of each stream appeared to be low but it helps in better utilization of the available network capacity. In some cases, this can even improve the overall throughput, especially when a single stream is limited by TCP window size or system constraints.

Loss and Jitter:

Packet loss and jitter were measured using UDP-based tests, as TCP inherently masks these effects through retransmission and error recovery mechanisms.

Since both the client and server devices were physically close and connected within the same subnet, the network experienced minimal delay and less congestion and inference. As a result, jitter (variation in delay) and packet loss were observed to be nearly zero, indicating a highly stable and reliable network environment.